



Using Recurrent Neural Networks To Solve Optimization Problems: Hopfield Neural Networks(HNN)

**By
Ekab-Osowo Samuel Tawo**

A dissertation submitted in partial fulfillment
of the requirement for the degree of
M.Sc. Data Science
(School of Engineering & Informatics)
The University of Wolverhampton
December 2022

TABLE OF CONTENTS

ACKNOWLEDGEMENTS	i
ABSTRACT	ii
CHAPTERS	
INTRODUCTION	1
1.1 NEURAL NETWORKS	1
1.1.1 How do Neural Networks Work	2
1.1.2 History of Neural Networks	2
1.1.3 Why Neural Networks	4
1.1.4 Types of Neural Networks	4
1.2 PROBLEM STATEMENT	5
1.3 OBJECTIVES OF RESEARCH	6
LITERATURE REVIEW	8
2.1 GROUPING OF PUBLICATIONS	8
2.1.1 Understanding Neural Networks	8
2.1.2 Understanding Hopfield Networks	9
2.1.3 Understanding Optimization Problems	9
2.1.4 Uses of Hopfield Networks in Optimization	10
2.2 RELEVANCE TO RESEARCH	12
USING RNNs IN OPTIMIZATION PROBLEMS (Application of Hopfield Networks)	14
SECTION A	14
3.1 RNN ARCHITECTURE	15
3.1.1 Standard RNN	15
3.1.2 Long Short-term Memory	16
3.1.3 Stacked RNN	17
3.1.4 Multiplicative RNN	17
3.2 LEARNING IN RNNs	18
3.2.1 Perceptron Learning Rule	19
3.2.2 Delta Learning Rule	19
3.2.3 Correlation Learning Rule	20

3.2.4 Out Star Learning Rule	20
3.3 APPLICATIONS OF RNNs	20
3.3.1 Practical Application of RNN	20
3.3.2 Theoretical Application of RNN	22
3.4 HOPFIELD NETWORK	23
3.4.1 Learning in Hopfield Network	24
3.4.2 Feedforward in Hopfield Networks	25
3.4.3 Energy Function of Hopfield Network	25
3.4.4 Associative Memory	25
SECTION B	26
3.5 METHODOLOGY	26
3.5.1 Data Collection Method	26
3.5.2 Hopfield Network Model Development	26
3.6 CONSTRUCTING A SIMPLE HOPFIELD NETWORK	27
3.7 IMPLEMENTATION	29
3.7.1 Tests and Results	29
3.7.2 Effectiveness of Model	31
3.7.3 Design Flow Chart	35
ANALYSIS AND DISCUSSION	37
4.1 ANALYSIS OF RESULTS	37
4.2 DISCUSSION	37
CONCLUSION AND FUTURE WORK	40
REFERENCES	41
APPENDICES	iii

ACKNOWLEDGMENTS

I would like to, first of all, thank my supervisor, Dr. Andrew Gascoyne, of the University of Wolverhampton for his commitment, patience, and time in providing quality and adequate feedback as well as aiding in the completion and success of this dissertation. I would also like to thank my family for their input and feedback on the success of this research and Uchechi Nwigwe for her constant help toward the completion and success of this dissertation.

ABSTRACT

Optimization problems are issues that exist in every aspect and scope of science and engineering. Such problems include but are not limited to, pattern and image recognition, linear, nonlinear, polynomial, and complex problems with various constraints all fall within the scope of optimization problems. Some important parts of soft computing are applied to fields such as logistics, resources, and sensor networks. Finding the best possible solutions is necessary when solving such issues; optimization problems need optimized methods. Optimized methods are characterized by the maximization or minimization of functions so the solutions needed are deemed valid[11].

This is where the study of Neural Networks has been highlighted and analyzed, aiming to solve various optimization problems. Recurrent Neural Networks specifically with the aid of certain learning rules have been applied in solving these nonlinear, multiple-constrained issues. It is important to solve these problems as it provides the best possible feasible solutions amongst other solutions in a given scientific scope. It also helps in minimizing errors. For this research, a specific Recurrent Neural Network type, Hopfield Network is used in pattern recognition. A Hopfield Network machine learning model is developed using the Hebbian Learning rule to attempt at solving optimization problems such as pattern and image recognition. The findings in this research lean towards the fact that the Hopfield Network model developed can to a large extent adequately recognize patterns however, some constraints still very much affect the accuracy of the predictive model.

CHAPTER 1

INTRODUCTION

The zeal to find the best possible solution amongst all viable solutions is something that is desirable in scientific and/or engineering complications. System identification, function estimate, and neural networks to name a few have been observed to be real-time solutions in solving optimization problems[4]. With the investigation of neural networks and their application in providing real-life solutions for optimization problems, comes knowledge of the various types of neural networks that exist. Feedforward neural network, a potent technique that can theoretically model any function with random accuracy providing good solutions is one way of solving optimization problems by providing good solutions. However, using larger, long-lasting contexts to help make better predictions involves adding network recursive connections to feedforward neural networks. This process turns feedforward neural networks into recurrent neural networks (a more competent and dominant neural network with the ability to complex non-linear sequences that have long time-range dependencies)[5]. Recurrent neural networks (RNNs) are intended to understand sequential varying patterns and with the increased importance of training due to the large datasets present, this neural network type with a lot of potentials is poised at providing the best solutions to real-time problems. Due to this, this research considers ways of solving optimization problems by training Hopfield networks (an RNN type) using the Hebbian learning rule an efficient and effective learning rule used by most RNNs. This rule involves learning by reinforcing connections amongst learned states[6].

1.1 NEURAL NETWORKS

Neural Networks are algorithms that deal with cognitive tasks; including learning and optimization processes that are vaguely based on ideologies relating to the nature of the brain. Mathematically, neural networks are also known as *directed graphs* that compromise the following;

- A state variable n is associated with all nodes i .
- A real-valued weight w is associated with each link (ik) between nodes I and k .
- A real-valued bias ϑ_i associated with node i .
- A transfer function $f_i[n_k, w_{ik}, \vartheta_i, (k \neq i)]$ defined for all node i , that defines the state of nodes as functions of its bias, weights of incoming links and connection by its links.

The nodes are known as *neurons*, links are *synapses*, and the bias is the *activation threshold*. The transfer function takes the form $f(\sum_k w_{ik} n_k - \vartheta_i)$, where $f(x)$ can be a discontinuous step function or a sigmoidal function where it's increasing smoothly. When a node has no links toward them, they are known as an *input neuron*, while *output neurons* are nodes that have no links leading away from them [1]. Neural Networks also involves building machines with the idea that these machines work similarly to how the human brain functions. This is done by using biological neuron-type components when developing these machines. It can also be said that Neural Network is a network where there's an interconnection between all its elements. Ideally, the major function of a neural network is to create an output pattern when an input pattern has been offered [2]. The name and structure of Neural Networks are a result of their imitation of the human brain as well as how biological neurons signal each other. These networks are reliant on training data with the aim of learning and improving its accuracy over a given period.

1.1.1 How do Neural Networks Work?

Understanding how Neural networks work begin after determining the input layer, and the allocation of weights that convey the importance of an assigned variable. The larger variables contribute considerably more to the output than other inputs. There are two types of variables largely considered in neural networks; discrete and continuous variables. The inputs are then multiplied by their respective weights and finally summed. The output is determined after passing through an activation function. Thus, if the output surpasses a particular threshold, activation of the node begins, and this passes data to the next layer that exists in the network. As such, it is said that the output of one node becomes the input of the corresponding node. Passing data from one layer to the next in a neural network show that the network is a feedforward network. To understand mathematically how neural networks work, we have to think of each node in the network as its model consisting of numerous attributes such as; weights, input data, biases, and output. Thus, the formula would resemble;[3]

$$\sum_{i=1}^m w_i x_i + \text{bias} = w_1 x_1 + w_2 x_2 + w_3 x_3 + \text{bias}$$

$$\text{Output} = f(x) = \begin{cases} 1 & \text{if } \sum w_1 x_1 + b \geq 0 \\ 0 & \text{if } \sum w_1 x_1 + b < 0 \end{cases}$$

1.1.2 History of Neural Networks

The idea of neural networks has been in existence for quite some time, with the first concept adopted in 1943 by Warren McCulloch and Walter Pitts [Mc43]. They recommended a universal theory of processing information that's based on networks of binary switching or

decision elements politely called *neurons*. These ‘neurons’ were smaller than the actual biological equals. Caianiello [Ca61] gave a “learning” algorithm allowing one to define the synaptic strengths of a neural network. The *mnemonic* equation, the algorithm by Caianiello features the basic principle of Hebb’s learning rule which would be discussed further in this dissertation. 1958 Frank Rosenblatt and some collaborators [Ro62] studied a particular type of neural network called *perceptron* because they thought it was a simplified biological mechanism model for processing sensory information. Perceptrons comprise two layers that represent the input and output layers as seen in (Fig. 1.0) below. The output layer’s neurons receive synaptic signals from that the input layer but not vice versa. Also, the neurons within a layer do not communicate with one another and due to the directional flow of information, a *feed-forward* network was birthed. Rosenblatt further led an iterative algorithm that helps in the construction of synaptic couplings (w_{ij}), resulting in a particular input pattern transmogrifying into the expected output pattern with the convergence also being proven. However, based on research done by M. Minsky and S. Papert [Mi69] where they attested to the fact that Rosenblatt’s group findings only worked for problems that could be solved by a perceptron. Furthermore, they showed the presence of unassuming problems that the two-layered perceptron can’t solve, and this can be seen in (Fig 1.1). The “exclusive-OR” (XOR) gate, requires input neurons (two) to be connected with a single output neuron in a way that the activating of the output unit occurs only when the input unit is active. The logic XOR gate is a standard problem that can be solved in computer design and this research from Minsky and Papert was a blow to the concept of the perceptron.

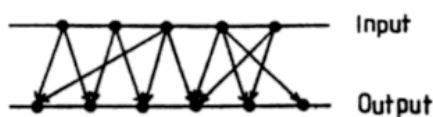


Fig 1.0

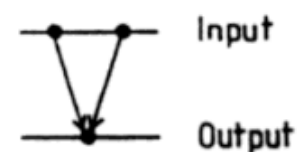


Fig 1.1

Feed-forward networks (perceptrons) have had a new interest in it, and this comes from the rediscovery of an algorithm that helps in defining synaptic coupling strengths in multilayered networks with hidden layers as can be seen in (Fig 1.2). Werbos [We74], suggested what is now known as the *error back-propagation* was recognized in 1985 by other scientists [Cu86, Pa85, Ru86a]. Their learning algorithm is built from the principle; those synaptic strengths (w_{ij}), are made iteratively in a way that the output signal varies as little as possible from the desired one. The gradient method helps achieve this principle (δw_{ij}). With the gradient method,

this is where the operation of the network corresponds with a highly nonlinear mapping between the input and the output - the step function in (Fig 1.0) is nonlinear - and this method is applied until convergence is reached. Error backpropagation is an example of a large class of learning algorithms called *supervised learning*. This is because, at each step of the way, the network is corrected by comparisons of the actual output with the desired output. These algorithms are not present in biological neural networks as there are only applicable when the desired output is known[1].

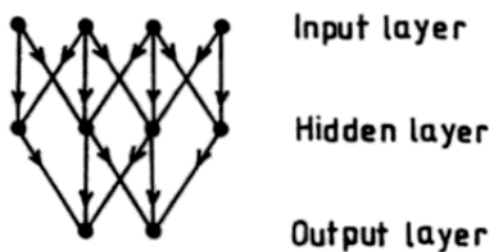


Fig. 1.3. A feed-forward network with a hidden layer.

1.1.3 Why Neural Networks?

The main function of neural networks is getting an output pattern when an input pattern has been presented to it. Neural networks are also used in pattern classifications, that is sorting patterns into groups[2]. Furthermore, interest in neural networks has also stemmed from the quest to understand underlying guidelines as to how the brain works. Also, the desire to build machines capable of achieving complex tasks which operatable and programmable computers aren't built for.

1.1.4 Types of Neural Networks

There are numerous types of Neural Networks with various attributes relatively different from one another. Each neural network type has its various uses and compromises.

The Perceptron – This is the oldest neural network having just one neuron and it is the simplest neural network form/type.

The Feedforward neural networks or multi-layer perceptrons (MLPs) – These compromise an input layer, a hidden layer(s), and an output layer. These neural networks consist of sigmoid neurons because of the nonlinearity of real-life problems. This is also the foundation for natural processing language (NLP, computer vision, and other neural networks as data is fed into these models for training.

Convolutional neural networks (CNNs) – These neural networks are usually used for image recognition, pattern recognition, and also computer vision. In identifying patterns in an image,

matrix multiplication is employed. These networks also pose similarities to the feedforward networks.

Recurrent neural networks (RNNs) – These neural networks are recognized for their feedback loops. They are used largely in predictions about future outcomes using time series data. These predictions can include stock market predictions, and sales forecasting to name a few. The recurrent neural network would be the focus of this project as it would be used to try and solve optimization problems [3].

This research is centered on using Recurrent Neural Networks and applying a specific type of RNN to solving Hopfield Networks. As such, understanding RNN is imperative for the success of this research and this can be seen in chapter 3, where an in-depth analysis of RNN is done. The chapter would also cover the types of RNNs, their architecture, and applications(practical and theoretical) amongst others. The following section highlights the problem statement of this research, identifying the research gaps and the necessary step(s) to move forward.

1.2 PROBLEM STATEMENT

An optimization problem involves finding the best solution among the possible ones available. Mathematically, it can be seen as a finite set of variables where the right values for the variables denote the optimal solution. If the variables range over real numbers, it is a continuous problem, however, if they take only a finite set of discrete values, that is a combinatorial problem[14]. Optimization problems need optimization methods that allow the maximization and minimization of various objective functions. Asides from the linear and polynomial examples of optimization problems, some problems that need optimization cannot be solved and as such, some form of approximation is necessary[12]. Generally speaking, optimization problems are nonlinear, and they are extremely complex due to numerous constraints. As such finding the best possible solutions requiring the shortest computation time is a goal set when solving optimization problems[13]. There is a need to find real-time solutions for optimization problems as these problems exist in scientific and engineering fields. Neural networks have over decades been used in investigating optimization problems such as system identification, filter design, function approximation, and regression analysis to name a few. The constraints and number of decision variables are large, and with the large constraints, there's a need for large-scale optimization methods. As such, neural network methodology can solve optimization problems in running times with large magnitudes more efficiently than other optimization algorithms[4]. This research is poised at looking at a specific recurrent neural network type (Hopfield Network) to solve optimization problems that occur within the scope

of pattern recognition and identification. Here, predictive modeling would be done where the model predicts based on the input given to it. Due to the challenges that exist during prediction, a learning algorithm(Hebbian learning) is applied to learn a predictive model(program) to potentially predict the data provided[15]. Research has been done into the use of the recurrent neural network to solve these problems specifically with global convergence as a unified method when developing optimization neural models. Irrespective of the progress being made, there are still several theoretical and practical issues that are unsolved within this scope.[4]. Optimization problems can come in numerous forms, as already established, exist in engineering and scientific problems to name a few. As such numerous consequences exist for not solving optimization problems. The primary or basic consequence of not tackling the optimization problem is not using the best possible solution when solving machine learning and technical issues at hand.

This research is poised at developing a predictive machine learning that returns an output based on the input received. This Hopfield Network model with the application of the Hebbian learning rule solely focuses on image and pattern recognition as that's the optimization problem being tackled in this research. The following section states the major objectives this research aims at achieving.

1.3 OBJECTIVES OF THE RESEARCH

With the understanding of what optimization problems are, how prevalent they are, and the importance of trying to solve them, as well as their presence in every facet of technology, this research is thus poised at achieving certain objectives to give more insights into solving these problems with the specified neural network approach.

- The first objective this research aims at achieving includes investigating and studying just how effective recurrent neural networks (Hopfield Networks) are in solving optimizing problems. This would be done by studying and researching already existing research on this topic with the aim of extracting relevant information and identifying research gaps in this field.
- The main objective of this research is to develop a predictive Hopfield Network model that can be used for predictive analysis and pattern recognition.
- Comparing the Hopfield network model with other RNNs may also be covered during the cause of this research.

- Finally, this research seeks to determine ways by which this model can be improved as well as provide necessary recommendations within this topic. In addition to this, contributing to Artificial intelligence specifically Neural Networks academia is part of what this research aims to achieve.

To begin achieving the objectives stated above, the next chapter gives a literature review of relevant publications that give insight into the scope of this research topic as well as provide important information on already existing models, their approaches, and conclusions.

CHAPTER 2

LITERATURE REVIEW

This chapter is focused on the relevant references used within the scope of this research. It highlights and gives insight into already existing research done in the neural network field. Highlighting their approaches, and conclusions to name a few. Besides from this, the importance of these sites to this research is also showcased. This chapter is outlined firstly by identifying what each research paper, book, and/or publication talks about. Followed by the approaches and conclusions discussed in the resources and finally their relevance to this project. The publications would be grouped accordingly to what resources were provided during the course of this research.

2.1 GROUPING OF PUBLICATIONS

The various resources used during the course of the research were grouped according to what subject matter covered. What each publication talks about, the approaches considered, and the subsequent conclusions derived are also highlighted in this section.

2.1.1 Understanding Neural Networks

The references highlighted here discuss neural networks in detail. Beginning from what a neuron is, [1] Gives insight into what neurons are, the definition and history of neural works, applications, and reasons why neural networks are necessary. Associative memory is also discussed. There are numerous types of neural networks as well as diverse angles by which neural networks can be understood. A different angle; pattern classification was highlighted in [2]. Here neural networks were defined via their application to pattern classification as well as highlighting specific uses of neural networks as well as why “neural networks” are “neural networks”. A more mathematical and computational approach is presented in [3] as they discuss the formulae involved with understanding how neural networks work as well as touching on the types of various types of neural networks. The mathematical equations, how they are derived, and the uses of these formulas in understanding neural networks are also highlighted. As already stated, there are numerous ways in understanding neural networks, and [4] does an extensive job of providing an understanding of neural networks highlighting. This book also gives insights, focusing on recurrent neural networks, their types, and architectures, as well as the application of recurrent neural networks and neural networks as a whole, also touching on the learning rules applicable to this network. This is also the first time the Hopfield network is mentioned. To understand recurrent neural networks, a major aspect to be covered

is their architecture. [5] provides in-depth knowledge into the various recurrent neural network architectures, their uses, and their importance as well as the numerous types of learning rules which exist, Hebbian, amongst others.

2.1.2 Understanding Hopfield Network and Hebbian Learning Rule

This section discusses already existing research done into Hopfield networks and Hebbian learning. [6] provides an introductory learning process into Hopfield networks analyzing the mathematical aspect of this network. Here, the energy function uses of matrixes, and their application is underlined. The Hebbian learning rule is mentioned here as the learning rule which is largely applied to numerous recurrent neural networks and Hopfield networks specifically. An application of the Hopfield network is also showcased, emphasizing its capabilities in pattern recognition. [7] is research done into understanding the Hebbian learning rule, its history, and basically how it is used? [8] extensively focuses on the Hopfield network. Here, the history, formulas, methods, approaches to understanding and using this network, and finally, associative memory is discussed. Still on how Hopfield networks work, [9], [10] gives insight into Hopfield network history as well as, calculations, and computations of the network type with the use of practical examples into its application. [30], gives insight into the various types of learning rules, their mathematical forms, and where each learning rule is applicable. It also highlights some differences between each learning rule and the Hebbian learning rule, the main rule used for computation in this research.

2.1.3 Understanding Optimization Problems

Optimization problems are the focus of this research. This research is poised at solving optimization problems and [11] involves research done into the application of neural networks in optimization problems. Highlighting specific types of optimization problems and how certain artificial neural networks are applied to solving such problems. Furthermore, [12], and [13] look at the new trends in the application of artificial intelligence in optimization problems, specifically power optimization. Insights into different types of optimization problems in different engineering and scientific fields were also discussed [14] highlighting the importance of optimization in machine learning and why it is important for optimization problems. Numerous research has been carried out into solving some types of optimization problems with some types of recurrent neural networks. [15] talks about the application of recurrent neural networks in chaotic systems via chaotic behaviors and dynamic neural networks. Recurrent neural networks are also applied to sequential processes and research did in [16] shows how these neural networks were applied in the bid to solve problems with linguistic errors. [17], [18] focused on recurrent neural networks and learning rules in the modeling of trajectory

systems. However, research done in [18], worked on modeling these systems without the use of incremental gradient descent as applied in [17]. Other areas where neural networks can be applied include complications that include long-term dependencies, looking at how advancements in NARX recurrent neural networks are beneficial to long-term dependency issues[19]. Neural networks are also applied in fuzzy logic networks[20].

2.1.4 Uses of Hopfield Networks in Optimization Problems

To carry out the major objective established in this research, it is imperative to understand the uses of Hopfield networks and their applications in solving optimization problems. This subsection highlights already existing research done into the application of Hopfield networks, as well as the approaches and conclusions, carried out in their methodologies. In[21], Hopfield neural networks(HNN) were applied to a distribution feeder recognition problem as this is an optimization problem for loss minimization. The approach carried out involved developing a network model that would be applied to the distribution system. In doing this, the energy function was minimized and turned into a neural network via the definition of a connection weight matrix as well application of numerous power flow equations. After implementing their model, they concluded that despite admiration for the Hopfield network as its application to similar optimization problems like the Travel salesperson problem, further research is required, as no valid solution was found. Similar research was carried out on the Economic load dispatch problem, an optimization problem in power systems. [26] developed an HNN model that was poised at minimizing the operating costs of systems while at the same time sustaining operating limitations. To do this, the weight factors were determined through the process of users determining the coefficients with the satisfaction of constraints. After implementation, it was deduced that a standard HNN model has capabilities that can aid in solving the Economic Dispatch problem. However, when solving this problem with larger constraints, the HNN model should be extended. Research in [24] was done into the application of Hopfield Neural networks (HNN) in the selection of mergers and acquisitions. A model which involved storing mergers and acquisitions standards of the acquirer as well as selecting the targets in conjunction with the requirements of the acquirer. This was done by switching the merger and acquisition standards of the acquirer into the binary threshold. The computation of the energy functions,(difference between the old and new) connecting weights, and network nodes were done. Upon implementation, the results showed that the HNN model was good in recognizing the targets according to the mergers and acquisition standards of the acquirer were accurately recognized. Furthermore, affirmations were made to the effectiveness of the HNN model and its pivotal role in decision-making concerning mergers and acquisitions. [27] addresses

Hopfield Neural Network being implemented into optimal CDMA multiuser detector. The model developed was an alternative multiuser detector and it is to be compared to other detectors in the synchronous transmission case. From the results, the HNN receiver outdid conventional detectors and is a powerful generalization of MSD which is presented using analog VLSI circuits. Also, the HNN proposed in this research is an eye-catching postprocessing stage that could improve the performances of other suboptimal unified multiuser detection schemes. [25] the research presents a Hopfield Neural Networks (HNN) model designed to produce a multi-scroll attractor by utilizing a new memristor as a synapse in the HNN. The goal of this research is to apply this multi-scroll memristive HNN to image encryption and this was done by constructing a new memristor with a hyperbolic tangent function before presenting it to the HNN model. Per the conclusion, the memristive HNN productively recreated distinctive dynamic behaviors that confirm the numerical simulation and analysis carried out in their research.

Since the resurgence of neural networks, there has been a well-known use in digital processing. Pattern recognition problems were often solved by linear, quadratic discriminants as well as non-parametric (k-nearest) neighbor classifiers. [28] analyzed major applications of neural networks in image processing as well as highlighting major strengths and weaknesses in image processing tasks. In approaching such a task, minimizing the energy function to converge at a stable state is required. It was deduced that the typical HNN model is ideal when solving problems including facial image recognition as well as digital communication problems. [29] develops a Hopfield Neural Network (HNN) model applying it to facial image recognition. The approach involved selecting seven facial images and reducing their sizes too (*60x60 pixels*) with also 45% distortion in the images. With the Hebbian learning rule being applied to store the weight matrix, bit-by-bit matching of the stable state facial image with the original is done. The results showed 100% retrieval for grey scale facial images that are of (*60 x 60-pixel size*) with up to 30% distortion for seven facial images. This proved that the Hopfield Network model can be used for face-based security applications. Still on the application of Hopfield neural networks on image-themed problems, [22] applied an unsupervised parallel approach, competitive Hopfield Neural Network (CHNN) to medical image segmentation while adapting the WTA (winner-takes-all learning mechanism). The approach adopted included minimizing the energy function, as well as developing a model that would dodge time-consuming procedures when generating values in the weight factors in the energy function. The model was compared to two other popular methods (the hard and fuzzy c-means) methods. The CNN model had satisfactory results as it effectively fulfilled the production of image segmentation

while network convergence was guaranteed. Furthermore, they deduced that the algorithm would also be effective in parallel implementation for real-time applications. [23] looks at how Hopfield networks are applied to the routing of computer networks. The approach involved defining the energy function with its minimization leading to its lowest state as this state would correspond to the routing solution. Their solution thus was the directed path from the source to the destination. After simulation, their proposed Hopfield network model to solve the routing problem was established with there being convergence to an optimal solution with a satisfactory percentage of error. [31] looks at the capacity of the Hopfield nets and how it affects training, learning, and retrieval errors.

2.2 RELEVANCE TO RESEARCH

This section covers the relevance of each resource and its importance to this research. From the methods and approaches carried out, this section shows how these are important to achieving the objectives of this research.

Understanding Neural Networks is a very important aspect of this research as it gives insight and a general understanding of what the research is trying to solve. One of the major objectives of this research is to solve optimization problems using a recurrent neural network(Hopfield network) which is a type of neural network. Thus, understanding the history, types, form, application, and dynamics of neural networks as a whole, before narrowing them down to recurrent neural networks and finally Hopfield networks is necessary. It also shows an upward trajectory in how the research was carried out. Publications that have done research into Hopfield networks and Hebbian learning are also relevant to this research/project as it emphasizes the major neural network type being used in this research to solve specific optimization problems. Thus, having a profound understanding of how the network works, the learning rules applied to it and even its application on practical examples is ideal for the effective and efficient progression of this research as well as its subsequent success.

General insights into understanding the diverse types of optimization problems in different fields of study are also relevant in trying to achieve the objectives set aside for this research. This can be seen in the resources that underscore various applications of Recurrent Neural Networks as well as Hopfield networks in solving optimization problems. Concerning the application of Hopfield networks, various models were developed to tackle the problem at hand. This is relevant to the research as it gives insight into how to develop models which can be used for this project. Also, conclusions and deductions made from each research carried out show the effectiveness as well as complications associated with working with, designing, and

developing the Hopfield Neural Network model. Learning how to develop an effective model to meet the objective set in this research can be gotten from these publications that focused on this research.

The next chapter is the main body of the research and this further detailed insights into most of the implementation, and methodology as well as an understanding of the topic and objectives that this research aims to achieve.

CHAPTER 3

USING RNNs IN OPTIMIZATION PROBLEMS (Application of Hopfield Networks)

This chapter focuses on achieving the main objectives of this research paper. As the “main body” of this research, the methodology, implementation, analysis, and discussion carried out throughout this study would be highlighted in this chapter. For better understanding, this chapter is divided into 2 sections (Sections A&B respectively). Section A covers the theoretical aspects and gives insights into understanding Recurrent Neural Networks, Hopfield networks, and Learning in Hopfield networks, with some examples. Section B details a more methodological approach to the implementation of the Hopfield network, how the model was designed, and how the model works.

SECTION A

Recurrent Neural Networks(RNNs) are designed to understanding and learning sequential as well as time-varying patterns. A focus of research after being introduced in the 1980s, this neural network type was poised at learning string characters. With examples such as Hopfield networks, BAM, recurrent backpropagation nets, and Boltzmann machine to name a few, [4] This sequence modeler uses recursive connections in storing past information. Due to the recursiveness of RNNs, which provides them with unlimited context, RNNs can make accurate predictions. RNNs have undergone optimization improvements enabling better performance rates when dealing with supervised learning tasks. These improvements have come in the form of amazing results in language modeling, speech, handwriting, and image recognition to name a few[5].

3.1 RNN ARCHITECTURES

The architecture of RNNs varies from fully interconnected networks to simple partially connected networks as well as multilayer feedforward networks that have unique input and output layers. The major difference between the two architectures is that fully connected networks have no distinct input layer of nodes, and each node gets its input from other nodes. Simple partially RNNs on the other hand, are also used to learn strings of characters with their nodes being either part of the feedforward structure or providing and receiving context and feedback from other nodes respectively. (Figures 1.4 & 1.5) highlight fully connected RNN and simple partially RNN respectively. The context units (C1 & C2) in (Fig 1.5) are processed like input units as they receive time-delayed feedback from the second layer units. Training

data have both inputs and wanted output. As stated before, simple partial RNNs can be used to learn string characters, in this case, the network is trained to predict the next letter in a string of characters. It can also be used in the validation of a string of characters.

Feedforward multilayer neural networks can have feedback added to them and this is done by two methods. The first method involves using the input values sequence (feedback from the hidden layer to the context pattern of the input layer) introduced by Elman. The second method by Jordan involves using an output values sequence (feedback from the outer layer to the context nodes of the input layer)[4].

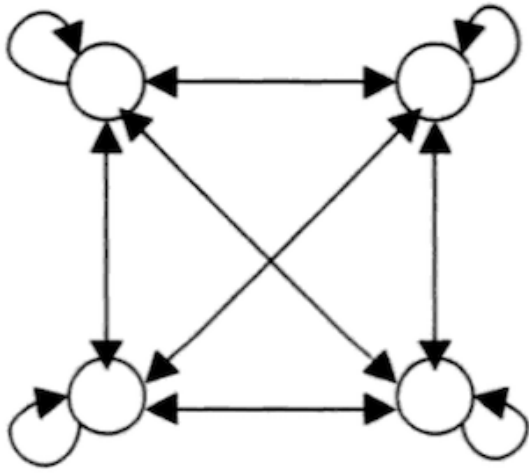


Fig 1.4

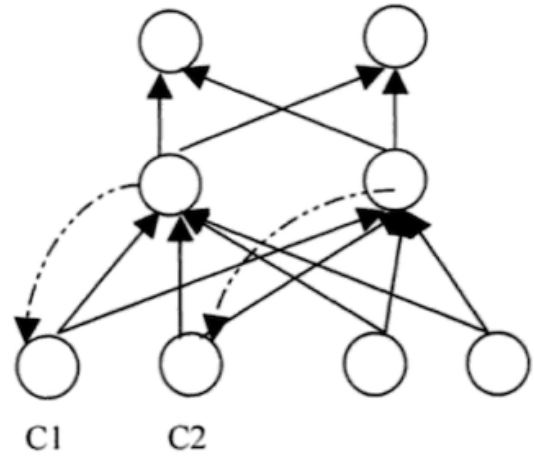


Fig 1.5

3.1.1 Standard RNN

Standard RNNs is an architecture that has an input vector I , a hidden vector H , and an output vector O . There exist 3 weight matrices; (W_{hi}) that are input vector to hidden vector, (W_{oh}) that is, hidden to output vectors and (W_{hh}) which is recurrently hidden to hide with a hidden state vector bias (B_h). The output units have softmax functions when functional and model a normalized probability distribution over the output units. The hidden units have the hyperbolic tangent ($\tanh$) applied to them. In the architecture, the RNN works through a sequence of T timesteps, predicting $O(t)$ using $I(t)$ and;

This can be seen in the equation and (Fig1.6);

$$H(t) = \tanh(B_h + W_{hi} I(t) + W_{hh} H(t-1))$$

$$O(t) = \text{softmax}(W_{oh} H(t))$$

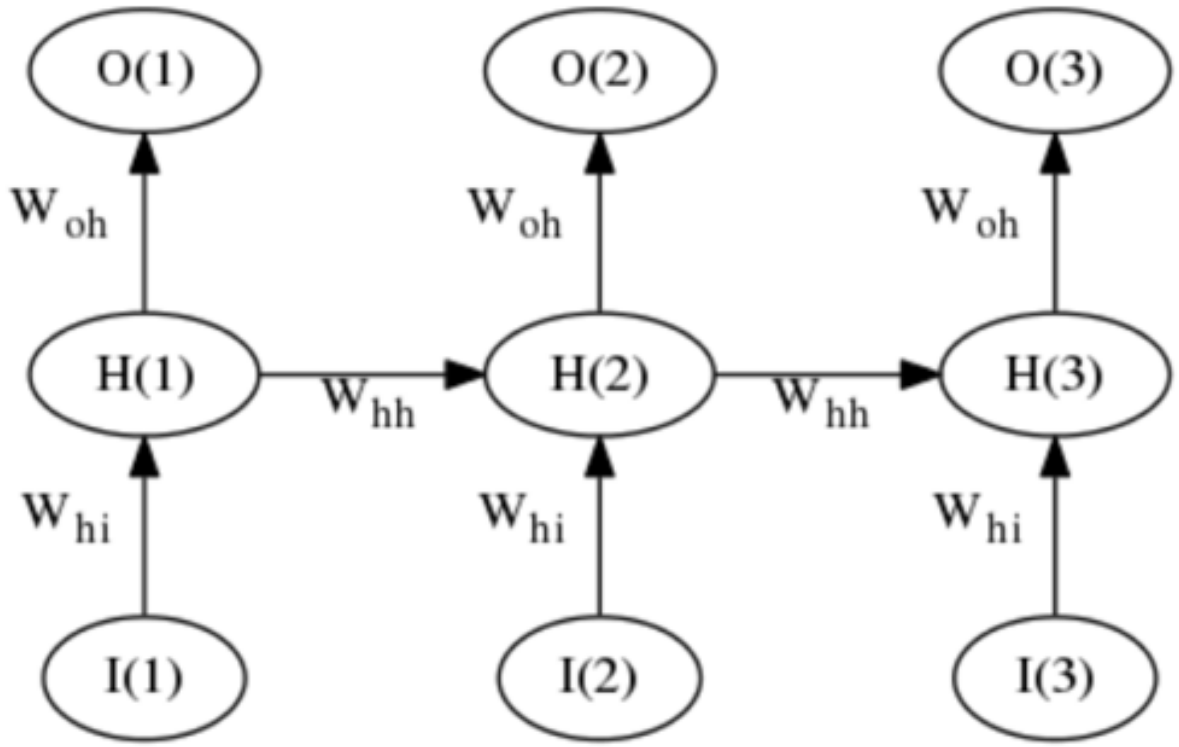


Fig 6

3.1.2. Long Short-Term Memory(LSTM)

Long short-term memory (LSTM) is another type of RNN architecture poised at tackling gradient-based learning issues that exist with standard RNNs. It tackles these problems by its memory cells using soft multiplicative gates to control information flows. Originally, LSTM input gates controlled the total input memory cells written to their internal state, while the output gates just what capacity of the memory cells' internal state is output. This can be seen in the formulas below; That is, the input gate controls how much is received from the input, and the hidden layer is written to the internal state of the LSTM gate having a self-recurrent weight of 1. The cells with aid from the input gate reject inappropriate inputs.

$$H_{state}(t) = H_{state}(t - 1) + \omega(t) \odot H_{in}(t)$$

On the other hand, the output gate which controls the internal state that is the output of the memory cells is compacted with a hyperbolic tangent. This output gate helps the LSTM cells to store information that may be irrelevant to the present output, but important later.

$$H_{out}(t) = \tanh(H_{state}(t) \odot p(t));$$

$$O(t) = \text{softmax}(W_{oh}H_{out}(t))$$

3.1.3 Stacked RNN

Stacked RNNs can be seen as a deep hybrid of recurrent neural network. There are used to increase an RNN's architectural expressiveness by “stacking” RNNs sequentially. This sequential stacking aids non-linear processing to happen between the input $I(t)$ and outgoing output $O(t)$. Also, it helps in allowing RNNs to accumulate diverse time scales of information at each layer. A limitation of this architecture is the occurrence of the gradient being backpropagated. This is done through feed-forward layers and time-augmenting discharge of vanishing gradient problems. In training stacked RNNs layer by layer training as done in deep neural networks is prevalent here. Below is a figure of basic stacked RNN.

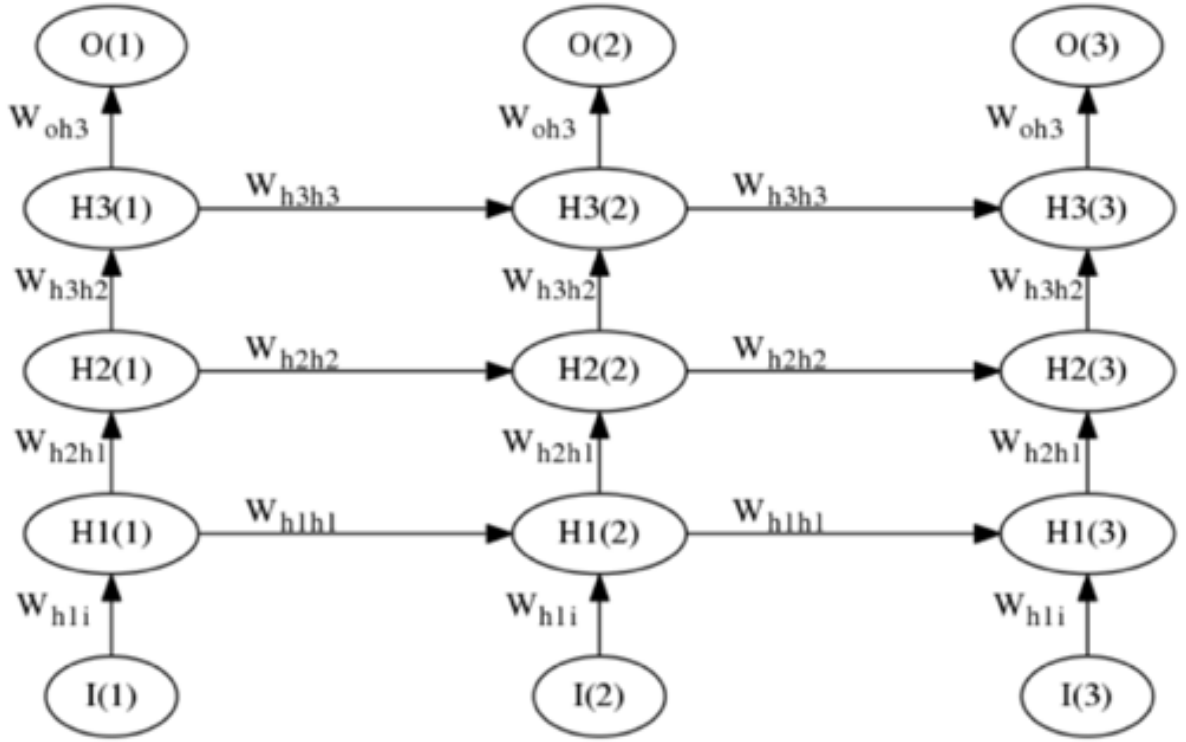


Fig 1.7

3.1.4 Multiplicative RNN

To permit varying transitions between hidden states dependent on their inputs, multiplicative RNNs were created. Multiplicative RNNs use a matrix factorization that allows weights in the hidden transition to change largely depending on the input. The hidden-to-hidden weight matrix (W_{hh}) in a standard RNN is then replaced through factorization with intermediate state M .

$$W_{hh} = W_{hm} \text{diag}(W_{mi} I(t)) W_{mh}$$

The effective hidden-to-hidden weight matrix that results from multiplying the output is different for each input $I(t)$. For example, if the signs of the effective hidden-to hidden weight are positive with some inputs and negative with others. The equation for this becomes;

$$H(t) = \tanh(B_h + W_h I(t) + W_{hm} \text{diag}(W_{mi} I(t)) W_{mh} H(t-1))$$

$$O(t) = \text{softmax}(W_{oh} H(t))$$

Due to the difficulty in training Multiplicative RNNs especially with gradient descent because of its high degree of curvature in its error surface, it is best to use 2nd order methods to train this architecture. Below is an image depicting multiplicative recurrent neural networks. (Fig 1.8) shows a multiplicative RNN.

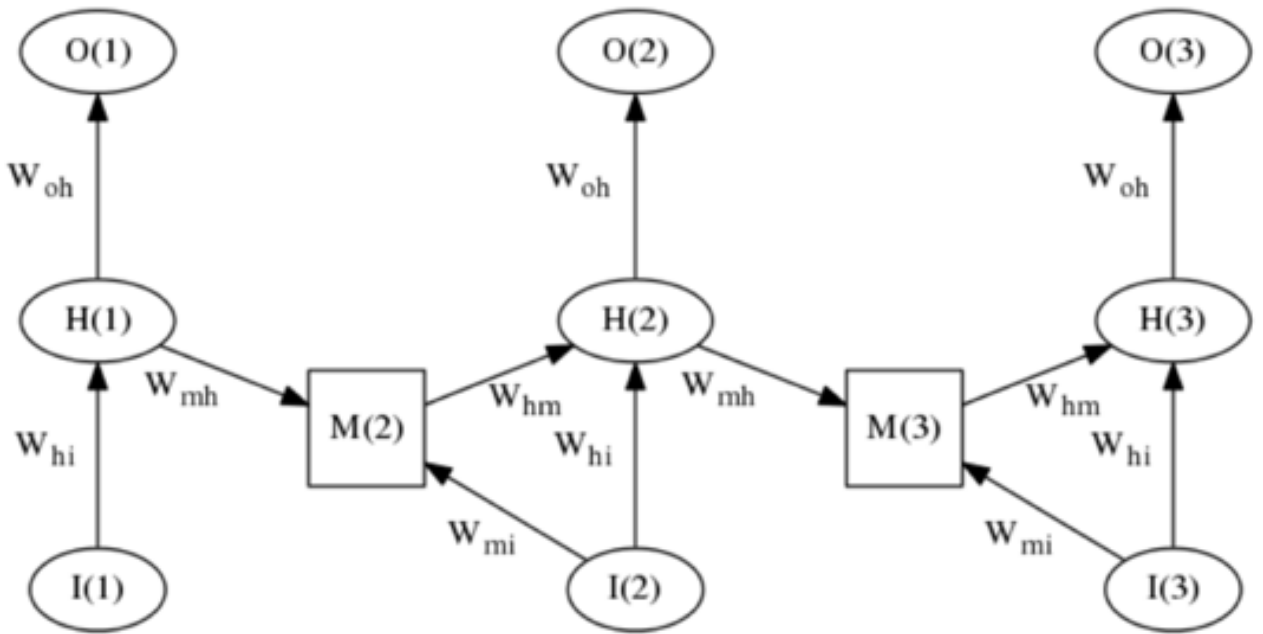


Fig 1.8

3.2 LEARNING IN RECURRENT NEURAL NETWORKS(RNNs)

“Learning is the ability of a machine to extract features from the supplied datasets to make its own representation”[7]. It is also an essential component of neural networks as it makes the approach attractive. This attraction relates to applications that have from time been a goal for artificial intelligence. Neural network concepts are largely based on Hebbian learning as well as gradient descent learning skills. Gradient descent learning is popular in back-error propagation introduced by Rumelhart and Werbos[4]. Hebbian learning, on the other hand, is

quite popular with Hopfield models as it is seen as how Hopfield models and its overviews store archetypes. That is, definite patterns are practiced just once to form the synaptic matrix[7]. Besides Hebbian Learning rule, other learning rules that are applied in RNNs include; Perceptron Learning Rule, Delta Learning Rule, Correlation Learning Rule, and Out Star Learning Rule.

3.2.1 Perceptron Learning Rule

This is an example of a supervised learning type, and its network starts by assigning random values to the weight. The output is then calculated based on a set of records that enables the expected output to be known. The output value is then compared with the expected output and the error function ϵ is calculated. The error function ϵ is also the sum of squares of errors that occur individually in the learning sample. This can be computed as:

$$\sum_i \sum_j (E_{ij} - O_{ij})^2$$

The first summation is done on each individual of the learning set, while the second summation is done on the output units. E_{ij} & O_{ij} are the anticipated and acquired values of the j th unit for the i th individual. The network then checks if there is an increase or decrease in the error function and this is done by adjusting the weights of the different units.

3.2.2 Delta Learning Rule

The learning rule depends on supervised learning and was developed by Widrow and Hoff. The rules state that modifying the synaptic weight of a node is the same as the multiplication of error and input. This is computed mathematically as:

$$\Delta w = \eta (t - y)x_i$$

For every input vector, the output vector is compared to the answer. If zero is the difference, learning hasn't taken place as such the weights are adjusted to reduce the difference. Weight changes are from u_i to u_j and that is $\Delta w_{ij} = \eta * a_i * e_j$, where:

- η is the learning rate,
- a_i is the activation of u_i ,
- e_j is the difference between the anticipated output and the actual output.

Delta learning rule can be used with single and several output units. During the application of the delta rule, it is presumed that the error is directly measured. The major objective in the

application of the delta rule lies in the reduction of the difference between the actual and anticipated output; the error.

3.2.3 Correlation Learning Rule

This rule has similarities based on the Hebbian Learning rule as it supposes that the weights between acting neurons are positive and weights between neurons with opposed reactions as negative. Unlike the case of the Hebbian learning rule, this is supervised learning rather than actual learning. The response, x_i , and desired response d_j are used in the weight-change calculation. Mathematically, the Correlation Learning Rule is:

$$\Delta w_{ij} = \eta x_i d_j$$

d_j is the preferred value of the output signal. In training, the algorithm starts with the initialization of weights to zero. This rule is also an example of supervised learning.

3.2.4 Out Star Learning Rule

This learning rule is used when the neurons are arranged in a layer. The weights are linked to a particular node which is expected to be equal to the desired outputs for the connected neurons via those weights. The out-star learning can be computed as [30]

$$w_{jk} = \begin{cases} \eta(y_k - w_{jk}) & \text{if node } j \text{ wins the competition} \\ 0 & \text{if node } j \text{ losses the competition} \end{cases}$$

This research aims to use Hebbian learning techniques on Hopfield models in predictive analysis and finding real-time solutions for optimization problems. As such, the Hebbian Learning Rule would be highlighted extensively in **Section 3.4**. Other learning techniques include Hessian free optimization an effective learning method that uses parallelization[5].

3.3 APPLICATIONS OF RECURRENT NEURAL NETWORKS

Recurrent Neural Networks can be applied to quite a several problems that exist within the engineering and scientific landscape. These problems include but are not limited to; issues in robotics, language learning, control systems, trajectories, and chaotic systems.

3.3.1 Practical Application of Recurrent Neural Networks (RNNs)

RNNs can be used in Chaotic Systems; within this scope of chaotic behaviors, dynamic neural networks are profound in having a variety of oscillations that can in turn help in understanding self-sustained activity patterns. Based on work done by [16], exploring a behavioral trend in a chaotic network when external patterns are being used as stimuli. This pattern stimulus appears

as a constant external input to all neurons in a single-layer recurrent network. A stimulus strength is diverse to the production of changes and trends in a complex oscillation. Stronger stimuli are more resilient to noise as well as evoking simpler oscillations. Per their examples, they show that a pattern-to-oscillation map can be used for pattern recognition (an activity that RNNs are exploited and researched to achieve). RNNs can also be used in Language Learning as per Kremer who assessed the relationship between language learning and recurrent neural networks, highlighting how formal language learning can aid in planning and employing recurrent neural networks. In the quest to understand this relationship, four lessons were observed; (1) Difficulty in RNNs training, (2) Learning is possible when search space is reduced, (3) Ordering of search space can accelerate learning, and (4) Ordering your training dataset can be helpful. Dynamic recurrent neural networks being presented with time-varying inputs are intended to decide outputs at certain points in time. Regarding this, a function mapping the input sequence to the output sequence of values is given to the issues where inputs are selected from a discrete alphabet of valid values while the output values stem into discrete categories. Recurrent neural networks help to categorize subsets of the input language, revealing efficient language learning techniques[4]. In Sequential Auto Association, where issues such as the development of proper linguistic representation need to be solved, recurrent neural networks can help in providing certain solutions. Global-memory recurrent neural networks; Jordan Recurrent Networks and Simple Recurrent Networks (SRN) have stirred the improvement of models that build representations of sequential inputs in its global memory. [17] presents an architecture developed to construct and process an ordered system of stationarily dispensed representation of complex sequential data. This comes from the ideology of building complex stationary representations of input sequence but has been extended to building these stationary representations in their form by developing rare representations for each input sequence. This model has components of Recurrent Autoassociative Networks (RANs), that learn to recreate input sequences as well as develop static dispersed representations of the sequence. RAN are recurrent neural networks that coincide with the dynamics of natural languages as well as being able to produce sequence representations and interpret them. This is done by unpacking them back to their sequential form. The model thus solves the problems associated with developing hierarchically structured sequence representation. Another essential application of recurrent neural networks comes in the modeling of systems that include trajectories (examples about precise required time relationships). [18], make comparisons of numerous recurrent neural network architectures and learning rules such as single-layer fully recurrent neural networks and multilayer networks.

Also, with incremental gradient descent, conjugate gradient descent, and three types of Kalman filter. From their conclusions, circle trajectory showed relative ease when learned but figure-eight trajectories proved difficult.[19], on the other hand, describe learning measures that do not require gradient evaluations. The performance of these algorithms comes from the use of complex spatiotemporal learning tasked with producing continuous trajectories. Distinguishing between two approaches where one uses the idea of learning automata theory and the other centered on the conventional simplex optimization approach. The training efficiency of the approaches was done with the spatiotemporal signal production using a trained neural network. This task with its complexity showed how capable recurrent neural networks are in approximating temporal dynamics.

Recurrent Neural Networks are also useful in robot control and learning. Ziemke in his chapter discusses and investigates the relevance of these robot control in AI and cognitive science. He highlights the unique second-order RNNs which are used in robot engineering detailing their capacities and how adaptive robot behavior works.

In control and signal processing algorithms, Recurrent networks are more powerful than nonrecurrent networks. Hagan, De Jesús, and Schultz introduced Layered Digital Recurrent Networks (LDRN), developing training algorithms and showcasing their application to control and signal processing problems. Presenting a necessary notation that represents LDRN, as well as discussing the dynamic backpropagation algorithm necessary to process training gradients for recurrent networks, their concepts underline the backpropagation-through-time and forward perturbation algorithm. These algorithms are presented in a unified framework and are exhibited for a simple, single-loop recurrent network[4].

3.3.2 Theoretical Applications of Recurrent Neural Networks(RNNs)

This subsection addresses the theoretical aspects of recurrent neural networks, by providing already established research done into it via knowledge representation, optimization, discrete-time systems, and long-term dependencies to name a few.

Gradient-descent learning algorithms for example are known to perform badly on tasks and complications that include long-term dependencies. These kinds of complications are those that involve having the expected output dependent on inputs that are from the past. However, an improvement in curtailing such issues comes from an advanced class of architecture called NARX recurrent neural networks which comprise strong representational capabilities. Regarding NARX, Lin, Horne, Giles, and Tino show that it doesn't bypass problems of long-term dependencies, however, just makes some improved performance on it[20].

The amalgamation of artificial neural networks (ANNs) with fuzzy logic (neuro-fuzzy systems) has become prevalent in numerous application domains. Giles, Thornber, and Omlin discuss this combination and further explained that the typical neuro-fuzzy model requires enhanced representational power for applications requiring context and state (knowledge representation). Based on their research, they provided synthesis methods that need fuzzy finite state automata (FFA) to go through some form of transformation before being mapped into recurrent neural networks. They finally prove the strength of fuzzy finite state dynamics that are constructed neural networks for finite values of network weight, which through recreations give empirical authentications for the proofs. This shows the knowledge equivalence representations that exist between neural and fuzzy systems as well as models of automata[21].

3.4 HOPFIELD NETWORK

As earlier stated, this research is poised at solving optimization problems with a specific recurrent neural network type; Hopfield Network. The Hopfield net is a recurrent neural network type that has a synaptic connection pattern where a fundamental Lyapunov (energy) function exists for the activity dynamics[8]. They can also be seen as sets of neurons that are bidirectionally linked to one another with symmetric weights[9]. Named after Dr. John Hopfield, this recurrent neural network type has units that are fully connected[10]. The Hopfield network is a type of recurrent neural network because the input of each neuron is the output of others. Fig 1.9 below shows the Hopfield network architecture consisting of 5 neurons. Fig 2.0, further highlights the recurrency of the network, highlighting its feedback loops.

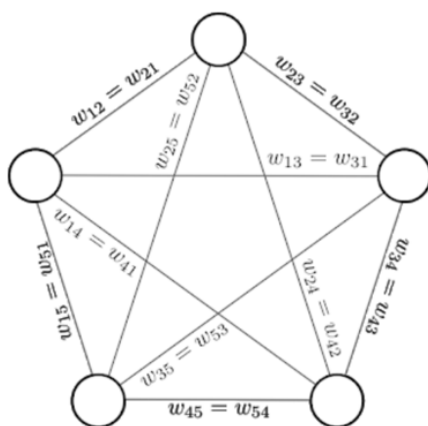


Fig 1.9

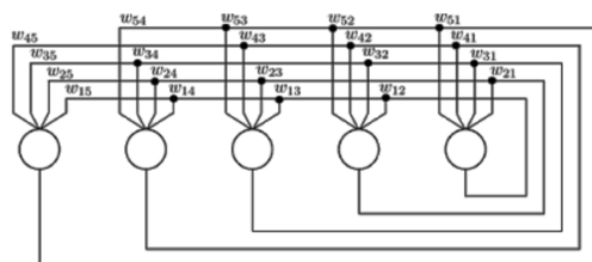


Fig 2.0

3.4.1 Learning in Hopfield Networks(Hebbian Rule)

Just like a most recurrent neural networks, Hopfield nets learn through the Hebbian rule which mathematically, indicates that the value of a weight (ω_{ij}) between 2 neurons (x_i & x_j) are the products of the value of that neuron. This is put as;

$$\omega_{ij} = x_i * x_j$$

The ideology behind this equation stems from the position that any two cells actively repeated at the same time would be “associated” making the activity in one enable the activity in another. e.g., let $x_i = 10$ and $x_j = 10$, then $\omega_{ij} = 100$, in the same instance;

$$\text{let } x_i = -10 \text{ and } x_j = -10, \text{ then } \omega_{ij} = 100$$

This example alludes to the fact that, if the neurons are the same value or have the same signs, they would have a positively large weight. However, having different signs would lead to the neurons repelling each other.[9] Hebbian learning rule was conceived by Cognitive Psychologist David Hebb, who stated that learning takes place through “reinforcing connections” amongst learned states.

In order to learn a pattern p ; where, $p = [p_1, \dots, p_n]^T$ where $p_{ij} = \pm 1$

A learning rate would be fixed $\varepsilon > 0$

The synaptic weights are updated by,

$$W_{ij}^{new} = W_{ij}^{old} + \varepsilon p_i p_j$$

After computation, $p_i p_j = 1$, as such Hebbian learning in matrix form is;[11]

$$W^{new} = W^{old} + \varepsilon(p \cdot p^T - 1)$$

The actual Hopfield Network, during iteration, the simulation would begin at an initial state;

$$X^{initial} = [X_1^{init}, \dots, X_n^{init}]^T$$

After which, selecting a neuron at random via the firing rule and updating is done. This is then repeated until a pattern is learned.

$$X_i^{new} = \sigma \left(\sum_{i \neq j} W_{ij} X_j^{old} - \theta_j \right)$$

3.4.2 Feedforward in Hopfield Networks

After learning has taken place in the network, the network is presented with a similar pattern to determine the output and in the case of Hopfield nets, the outputs are reconstructed patterns. Feedforwarding in Hopfield nets happens by initializing the neurons with values of a pattern after which computation of the weighted sum of each neuron takes place. In a place whereby the sum and the value of the neuron do not have the same sign, the neuron is flipped, hence nothing is done when they have the same sign. This process is, allowing the network to evolve toward a learned pattern.

After assuming the values of each neuron to be +1 or -1, the weighted sum for any neuron a_i ;

$$Z = \sum_{\forall j \neq i} \omega_{ij} x_j$$

The values of all the neurons x_j connecting to a_i would all be multiplied by ω_{ij} , between itself and x_i after which a summation of all the values is done. Concerning Hopfield networks, the value of the sum is irrelevant. The important feature is the sign on the sum.

As such, the value of that neuron would be;

$$x_i = \text{sign}(z)$$

The value of the neuron changes to match the sign of the weighted sum. This is repeatedly done until it comes to a stable state. A stable state in this context is when the sign of any neuron matches the sign of its weighted sum.

3.4.3 Energy function of a Hopfield Network

The energy of a Hopfield network is the summation of each neuron multiplied by its weighted sum. Due to the symmetry of the weights, it is divided by 2. In Hopfield nets, there is no labeled training set, patterns are watched and then memorized. If a network comprises N neurons, the energy function would then be n-dimensional. The computation for the energy E is as follows;[9] Each time a neuron fires, the energy E decreases[11].

$$E = -\frac{1}{2} \sum_i x_i \left(\sum_{j \neq i} \omega_{ij} x_j \right)$$

3.4.4 Associative Memory

An information storage device is said to be an **associative memory** when it allows the recollection of information on the foundation that it has limited knowledge of the content

without knowing where the content is stored. Associative memory is the storage and recollection of information via association with other information. It is a neural network's easiest use of collective computation[1].

Understanding the process by which feedforwarding and learning(Hebbian rule) in the Hopfield network is very pivotal to the success and progress of this research. The computation in this chapter paves the way, giving insights into the approach by which the expected predictive model would be developed. The next section is poised at providing the approaches taken to develop the predictive model. The major variables, significant themes, techniques, and necessary materials to develop the expected model as well as how the model would be/was developed would be discussed.

SECTION B

As stated earlier, this section focuses on the methodological approaches which were carried out during the duration of this research. Implementations as well as some analysis would also be highlighted. This section would be divided into two parts (Part 1 & 2), which would emphasize the general methodology, and the development of the model, with Part 2 focusing on implementations(tests and experiments).

PART 1

3.5 METHODOLOGY (APPROACH)

For this research, a mixed methods approach is observed. This is because the expected conclusions are expected to be both scientific and subjective. This is based on the particular type of research being conducted in this project. The results may both be statistically measurable as well as in words.

3.5.1 Data Collection Method

The research data type being used would be simulation data. These data types are generated based on real-world systems and are tested using a computer test/algorithmic model just like the proposed Hopfield Network model. This method type determines the possible reasons, factors, and/or constraints that can affect what can, could, would, and should happen during the course of the project.

3.5.2 Hopfield Network Model Development

A machine learning algorithm in Python programming language is to be developed to predict and recognize patterns. This model (Hopfield Network) is to receive a set of the input image(s), and stores them, after which a guess image is presented to the model. The model is to go

through its “memory/stored image(s)” and determine/predict an output image from its memory upon recognizing via the guess image. This is easier said than done, and to begin understanding the development of the model, it is crucial to first highlight mathematically, the various means by which the algorithm can and should be written. The Hopfield network like all Neural Networks is largely understood by understanding neurons and how to apply them. The development of this model, where a pattern is to be learned and then the application of Hebbian learning, can be adequately understood mathematically below. In the Hopfield Network, positive (+1) means the neuron is fired or is on and negative (-) the neuron is off.

Let’s take an example to adequately highlight the Hebbian learning as well as the Hopfield network. We assume $N = 3$ neurons with values of x_1, x_2, x_3 , to be $[1, -1, 1]$ respectively, then; We get to learn the pattern and update the synaptic weights $p = [p_1, \dots, p_n]^T$ where $p_{ij} = \pm 1$. A learning rate would be fixed $\varepsilon > 0$ and,

the synaptic weights are updated by, $W_{ij}^{new} = W_{ij}^{old} + \varepsilon p_i p_j$

$$p \cdot p^T = \begin{bmatrix} p_1 & 1 & p_1 p_1 & p_1 p_2 & p_1 p_3 \\ p_2 & 1 & p_2 p_1 & p_2 p_2 & p_2 p_3 \\ p_3 & 1 & p_3 p_1 & p_3 p_2 & p_3 p_3 \end{bmatrix}$$

After computation, $p_i p_j = 1$, as such Hebbian learning in matrix form is; [11]

$$W^{new} = W^{old} + \varepsilon(p \cdot p^T - 1) = \begin{bmatrix} 1 & -1 & 1 \\ -1 & 1 & -1 \\ 1 & -1 & 1 \end{bmatrix}$$

$$Z = \begin{bmatrix} Z_1 \\ Z_2 \\ Z_3 \end{bmatrix} \text{ Therefore, } Z = \sum_{j \neq i} \omega_{ij} x_j$$

Continuing with the example, the weighted sum (Z) for the three neurons is computed as;

$$Z_1 = \omega_{12} x_2 + \omega_{13} x_3 = (-1)(-1) + (1)(1) = 1 + 1 = +2$$

$$Z_2 = \omega_{21} x_1 + \omega_{23} x_3 = (-1)(1) + (-1)(1) = -1 + (-1) = -2$$

$$Z_3 = \omega_{31} x_1 + \omega_{32} x_2 = (1)(1) + (-1)(-1) = 1 + 1 = +2$$

$$x_1 = \text{sign}(z_1) \text{ which is, } (+)$$

$$x_2 = \text{sign}(z_2) \text{ which is, } (-)$$

$$x_3 = \text{sign}(z_3) \text{ which is, } (+)$$

3.6 CONSTRUCTING A SIMPLE HOPFIELD NETWORK MODEL

The Hopfield Network model would be developed using Python programming language with the Tkinter (“TK Interface”) package on the platform. In the beginning, we assume $N = 4$ neurons

values of x_1 , x_2 , x_3 , & x_4 , to be [1,1,-1,-1]. The weights are then updated and the Hebbian matrix form is also computed. The weighted sum (Z) is also computed and as expected, the signs being the important aspect of the equation is seen. Fig 2.1 below shows the neurons and upon calculation the signs of the weighted sum. The computation (python code) can be seen in Appendix I.

```
[[ 1  1]
 [-1 -1]]
Transpose Function
[[ 1  1 -1 -1]
 [ 1  1 -1 -1]
 [-1 -1  1  1]
 [-1 -1  1  1]]
Identity Function
[[1.  0.  0.  0.]
 [0.  1.  0.  0.]
 [0.  0.  1.  0.]
 [0.  0.  0.  1.]]
Weight
[[ 0.  1. -1. -1.]
 [ 1.  0. -1. -1.]
 [-1. -1.  0.  1.]
 [-1. -1.  1.  0.]]
Weighted Sum
[ 3.  3. -3. -3.]
Sign
[ 1.  1. -1. -1.]
```

Fig 2.1

Fig 2.1 above, shows the original matrix of the four (4) neurons (2:2). The positives are fired neurons and the negatives are not fired. Based on the formula as seen in section 3.5.2 above, the Transpose function; $p.p^T$ depicts each neuron being multiplied by itself. Then, the new weight matrix, upon updating produces the weight function as seen in the image above. The weighted sum can also be seen; that is the calculation for Z. The signs of x_1 , x_2 , x_3 , & x_4 , are also present. This computation would be the bedrock for the model being developed.

PART 2 – GRAPHICAL USER INTERFACE

This part as stated above, focuses on the implementation, highlighting the various tests and experiments carried out during the course of the research.

For simplification and user-friendliness, when implementing the model, the Hopfield Network would be developed with the aid of a Graphical User Interface (GUI). This interface(a grid) depicts white boxes as the showcase of neurons in the network. When a white box is clicked, this implies a neuron is being fired/activated or a neuron is positive. A positive neuron is depicted by the white boxes being clicked changing to blue in color and a negative, not clicked, or deactivated neuron remains white. Fig 2.2 below shows the user interface for a 9×9 matrix that is 81 Neurons Fig 2.3 depicts when neurons have been activated, in this case, clicked. The next part shows an implementation of the model to see if it achieves the objective set for this research. When a user runs the model, it signals the user to store the select number of images which has been issued in the program. This helps the user know the number of input images/patterns that are required as well as the guess image/pattern. After the user has followed the instructions on the screen and carried out the task, the output image/pattern is returned. All codes relating to how the grid and the model were developed are located in Appendix II

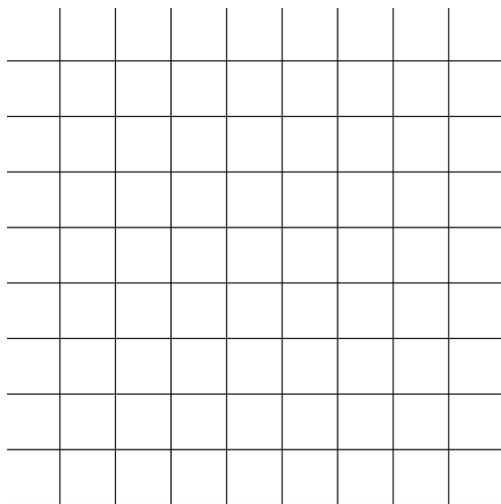


Fig 2.1

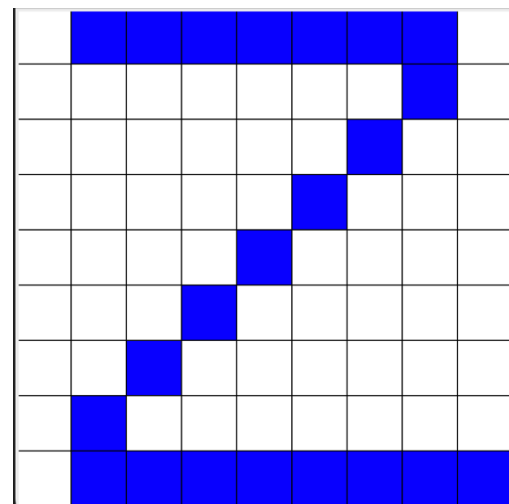


Fig 2.2

3.7 IMPLEMENTATION

The tests carried out help to determine if the model works, and then experiments would highlight just how effective the model is. The codes used to generate each test are seen in the appendix.

3.7.1 Tests and Results.

Test I - Assuming we give the model an image (1) to store as an input, we then provide a “guess image/pattern”(similar to the input), the expected output/result is for the model to accurately predict and/or return the correct input pattern/image. Images below show the input pattern, the

guessed pattern, and the output image/pattern. This test code as well as the various matrixes would be seen in Appendix III. Table 1.0 shows the results.

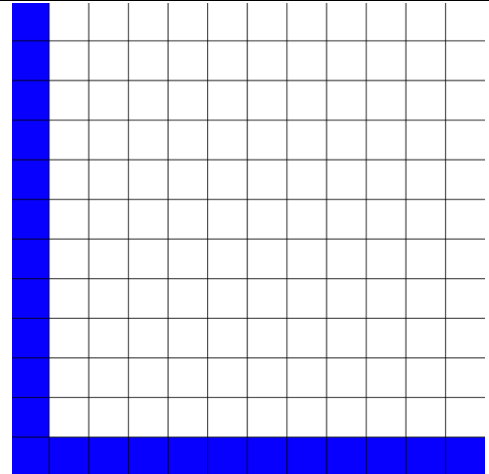
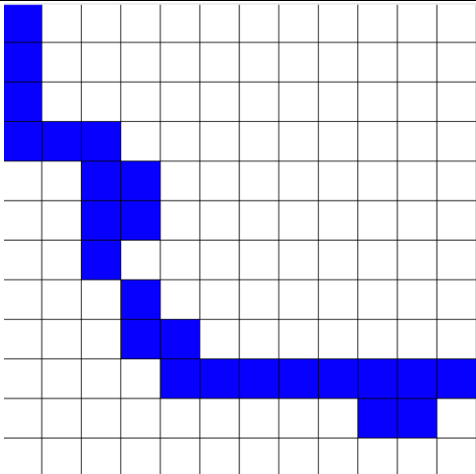
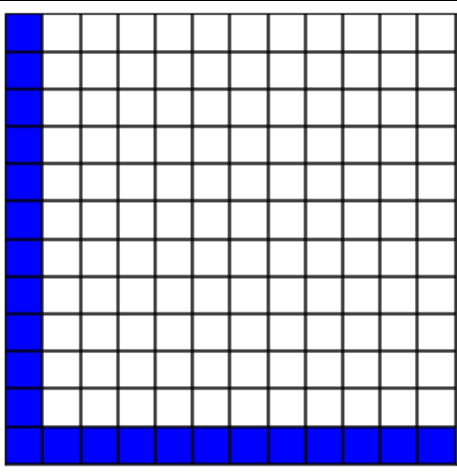
Input Pattern	Guess Pattern	Output Pattern
		

Table 1.0

Test II - A similar test of one image/pattern being inputted, this time, a guess pattern not similar to the input. The hope is for the model to return the expected output. Table 1.1 below shows the results.

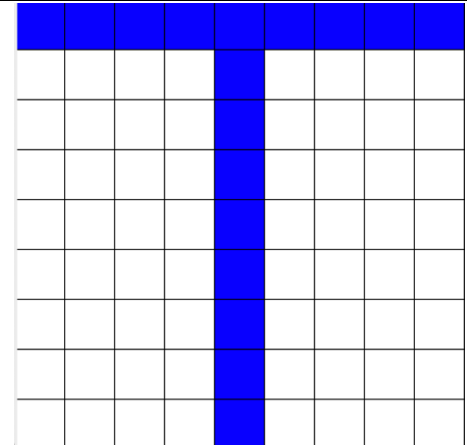
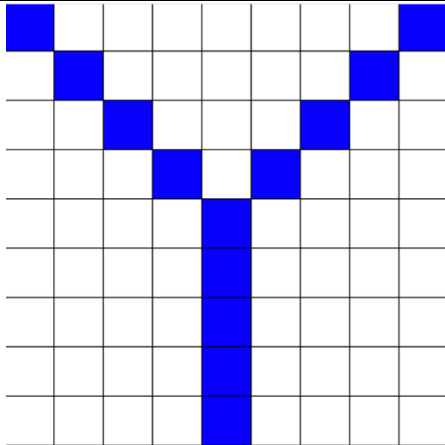
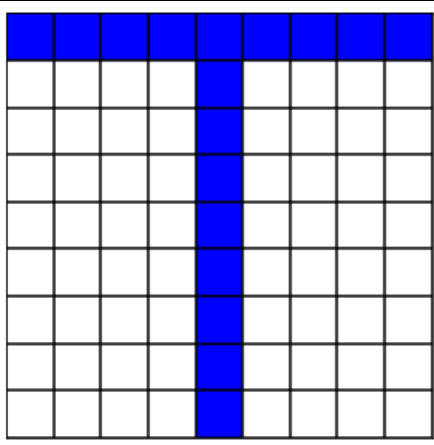
Input Pattern	Guess Pattern	Output Pattern
		

Table 1.1

Test III – An increase is done in this test, as three images are inputted and furthermore stored in the model’s memory. A guess image is also presented to denote if the model can accurately present the expected output from the images stored in its memory. This would be the first test involving an increase to the stored images being saved in the memory of model. Table 1.2 below shows the input/ stored images/pattern. Figs 2.3 & 2.4 show the guess and output images/patterns respectively.

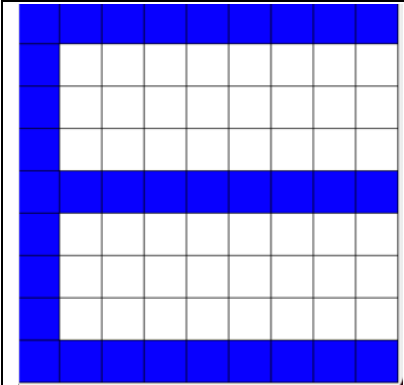
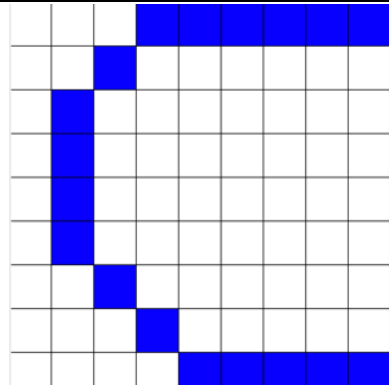
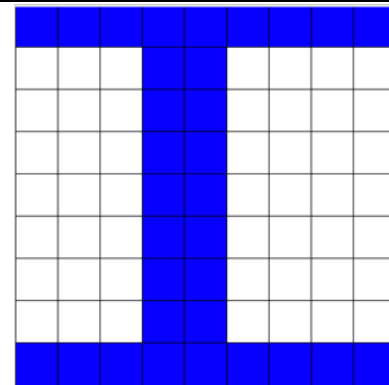
		
---	---	---

Table 1.2

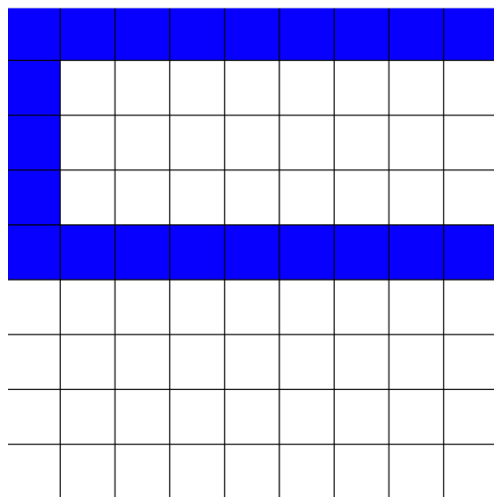


Fig 2.2

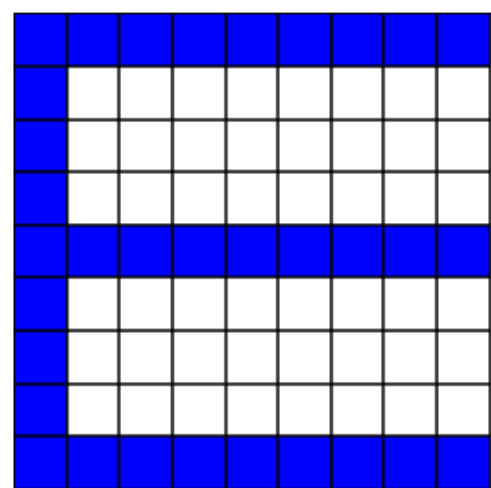


Fig 2.3

3.7.2 Effectiveness of Model (Experiments)

A machine learning or algorithmic model can satisfy the basic needs which it was developed for. However, to be certain that the model is efficient, it is important to see just how effective the model is by testing its effectiveness. To check the effectiveness of the model, a series of experiments would be carried out which would ascertain how reliable the developed model is. The experiments carried out in this section vary from seeing the maximum number of images/patterns that can be stored in the model; to if there is a maximum. Also trying out empty grids what the model return amongst other would be highlighted in this subsection.

Experiment I - For this experiment, the model was tasked with storing two images/patterns which are seen below (input patterns). Which one of the patterns is an empty grid? Figs 2.4 % and 2.5 are the input patterns while Figs 2.6 & 2.7 are the guess and output patterns/images respectively.

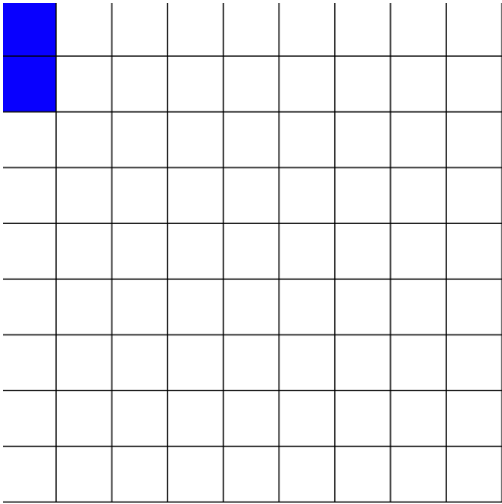


Fig 2.4

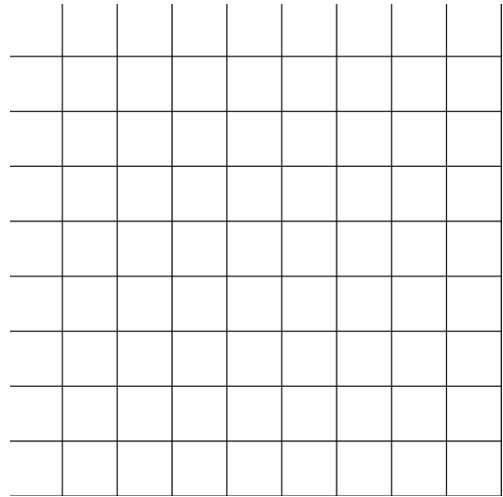


Fig 2.5

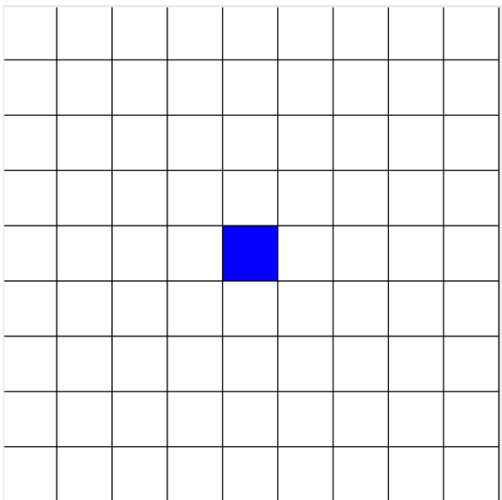


Fig 2.6

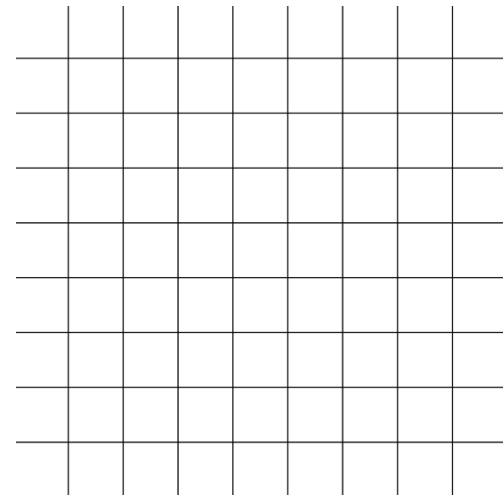


Fig 2.7

Experiment II - An empty grid as the guessed pattern in Fig 3.0. Fig 2.8, Fig 2.9 & 3.1 shows this.

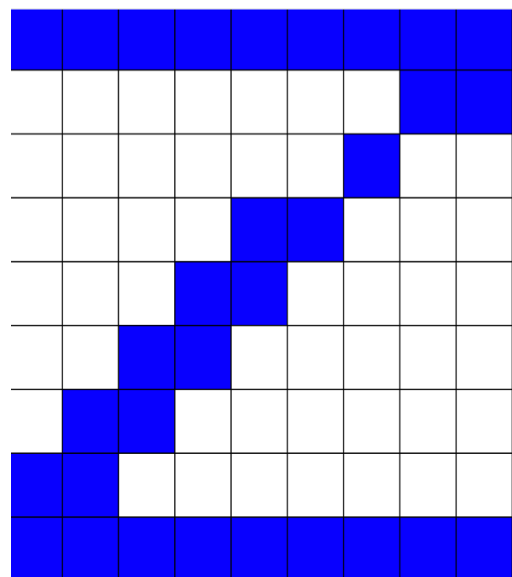
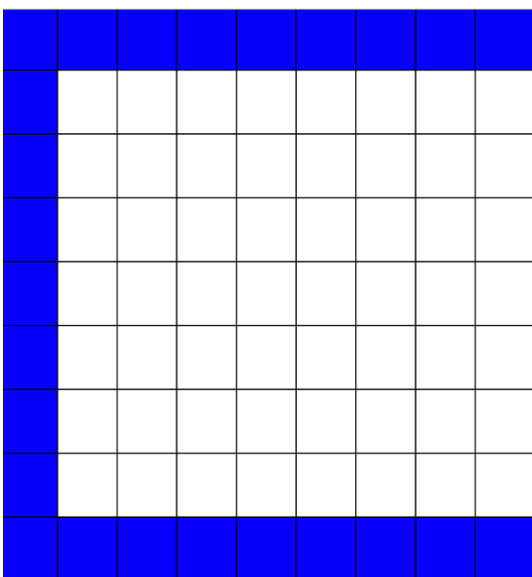


Fig 2.8

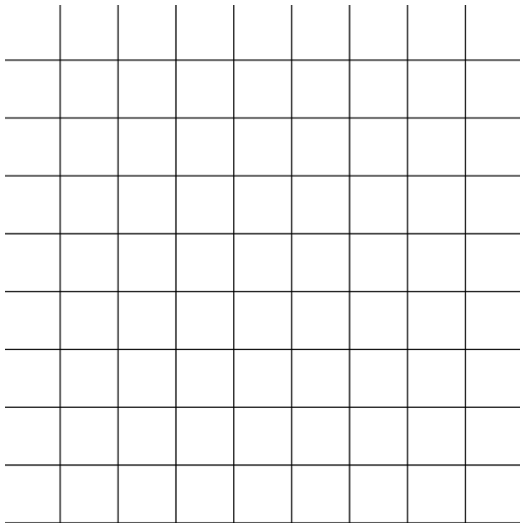


Fig 3.0

Fig 2.9

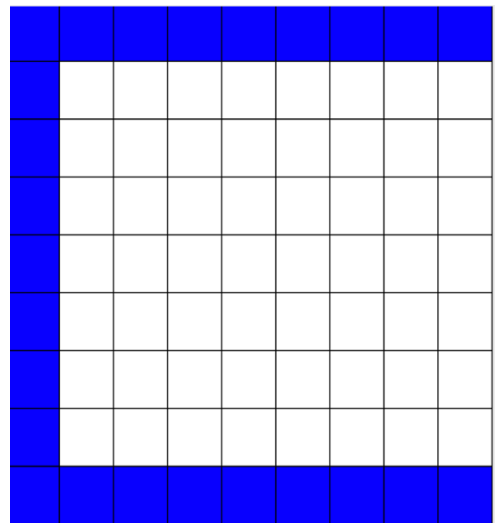


Fig 3.1

Experiment III - For this experiment, I attempt to increase the number of images that can be stored in the memory of the model. Ten (10) images were stored all with different patterns as well as the inclusion of an empty grid, and a guess pattern. The same was attempted with fifty (50) images/patterns stored, some as empty grids but most with patterns. The output pattern can be seen below in Fig 3.1.

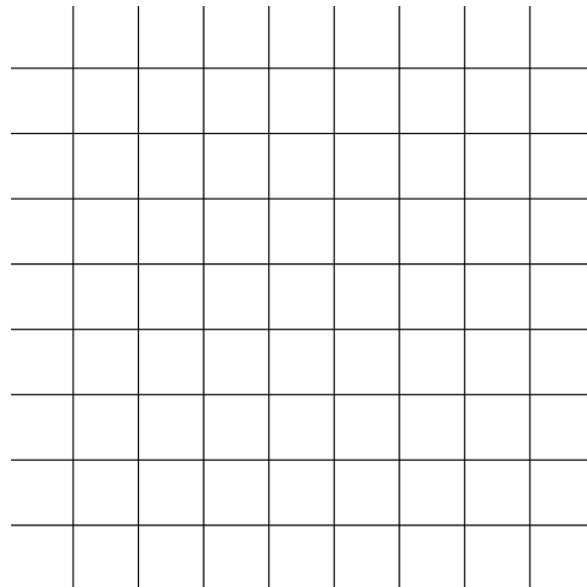


Fig 3.2

Experiment IV - An experiment similar to *Experiment II*, without an empty grid, is performed here. The model also takes and stores two (2) images/patterns, with a guess pattern present to

see what the model returns. This is different from the tests, as, with the tests, only one image/pattern was stored with a guess pattern and an expected output. Figs 3.3 & 3.4 are the input patterns, with Figs 3.5 % and 3.6 being the guess and output patterns respectively.

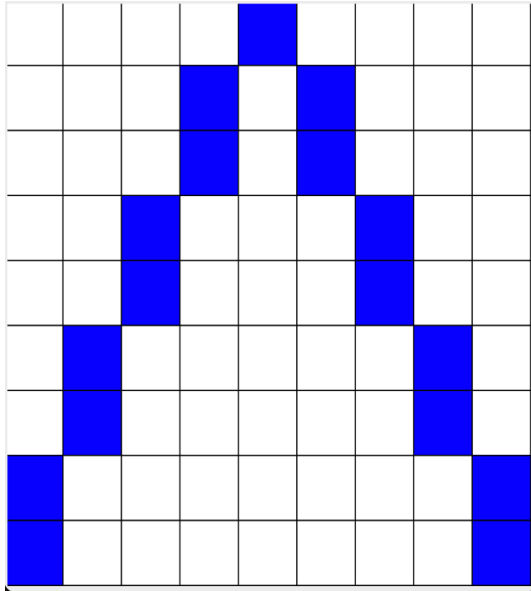


Fig 3.3

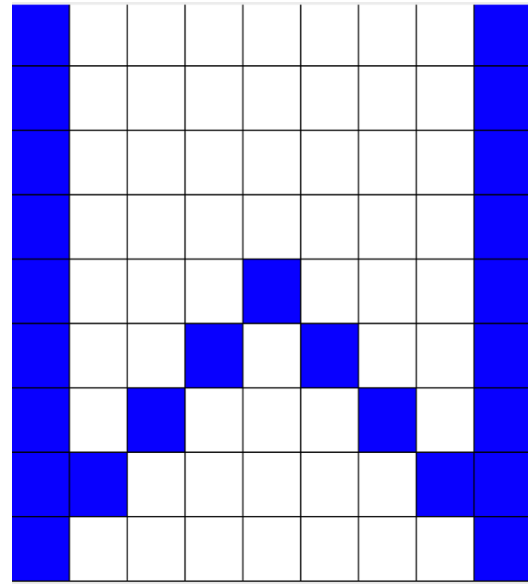


Fig 3.4

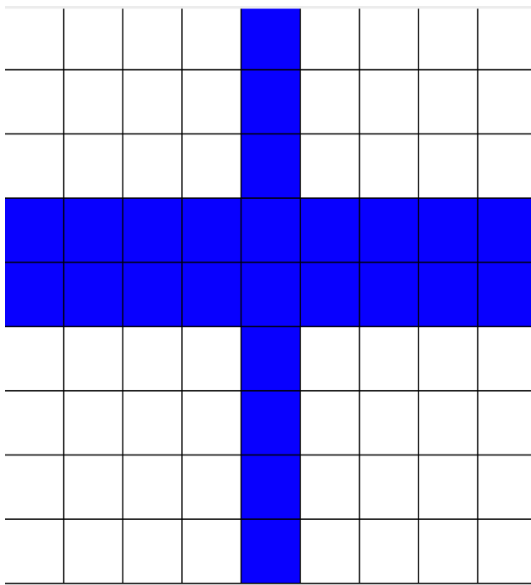


Fig 3.5

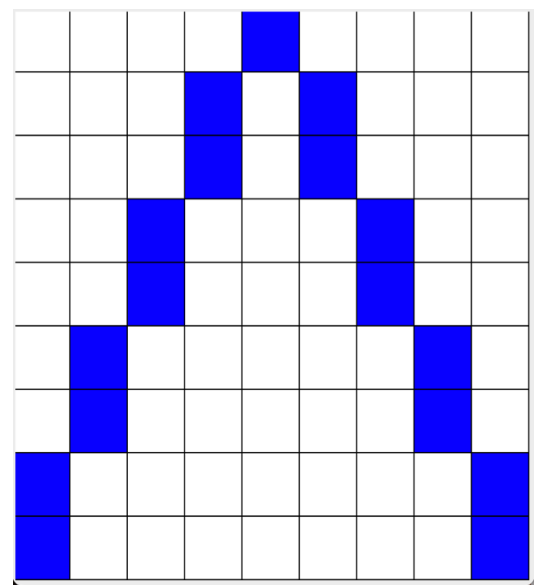


Fig 3.6

Experiment V - The final experiment involved storing 8 different images/patterns, and then having a guess pattern different from the 8 stored images to see what the model returns. The input patterns are in Table 1.3 below. Figs 3.7 & 3.8 are guessed and output patterns respectively.

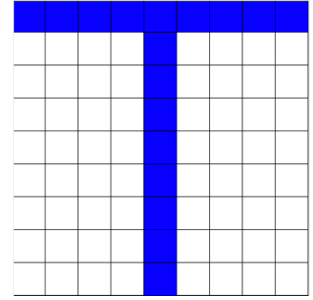
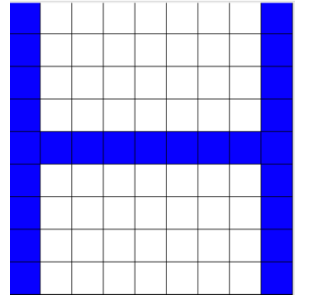
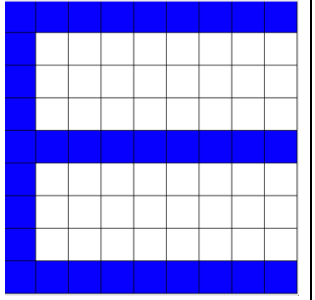
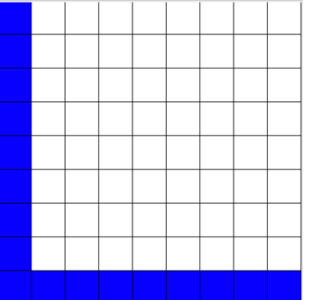
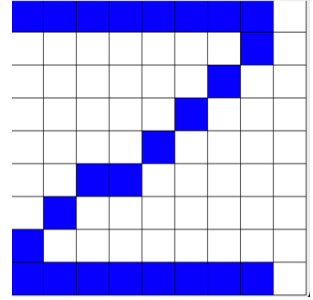
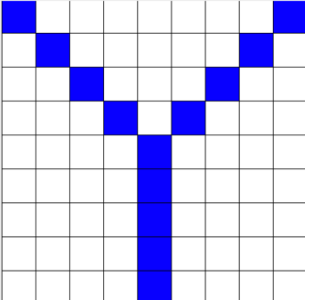
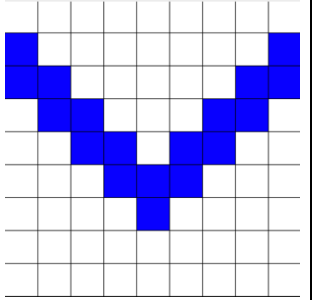
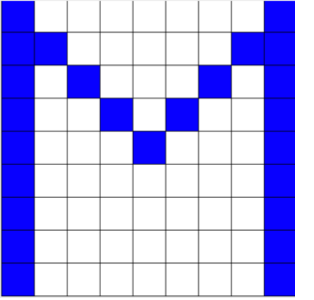
			
			

Table 1.3

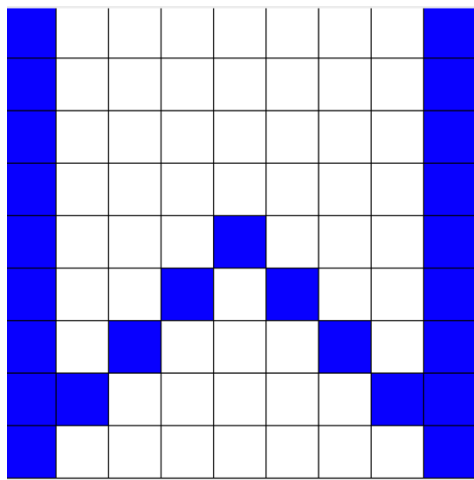


Fig 3.7

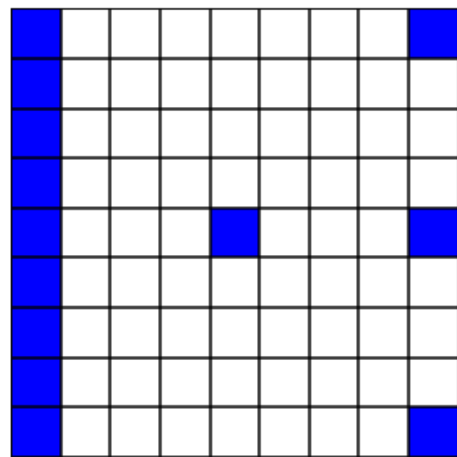


Fig 3.8

3.7.3 Design Flow Chart

This section gives a flowchart as to how the development of the model was carried out during the research. The first step involved understanding the neural network that was going to be developed, that is Hopfield Network. Using a 2×2 matrix (4 neurons), the Hebbian learning was applied to the network and the model was first computed manually, before being developed in the python programming language. The next steps involved developing a graphic user interface (GUI), specifically with a 9×9 matrix (81 neurons). Multiple iterations took place for the model to learn the pattern and to be able to recognize each pattern. This was testing, with numerous experiments performed to check the effectiveness of the model before analysis of

the results was done. Finally, a suitable conclusion regarding how effective the model was if the main objectives were met was deduced from the results gotten after testing.

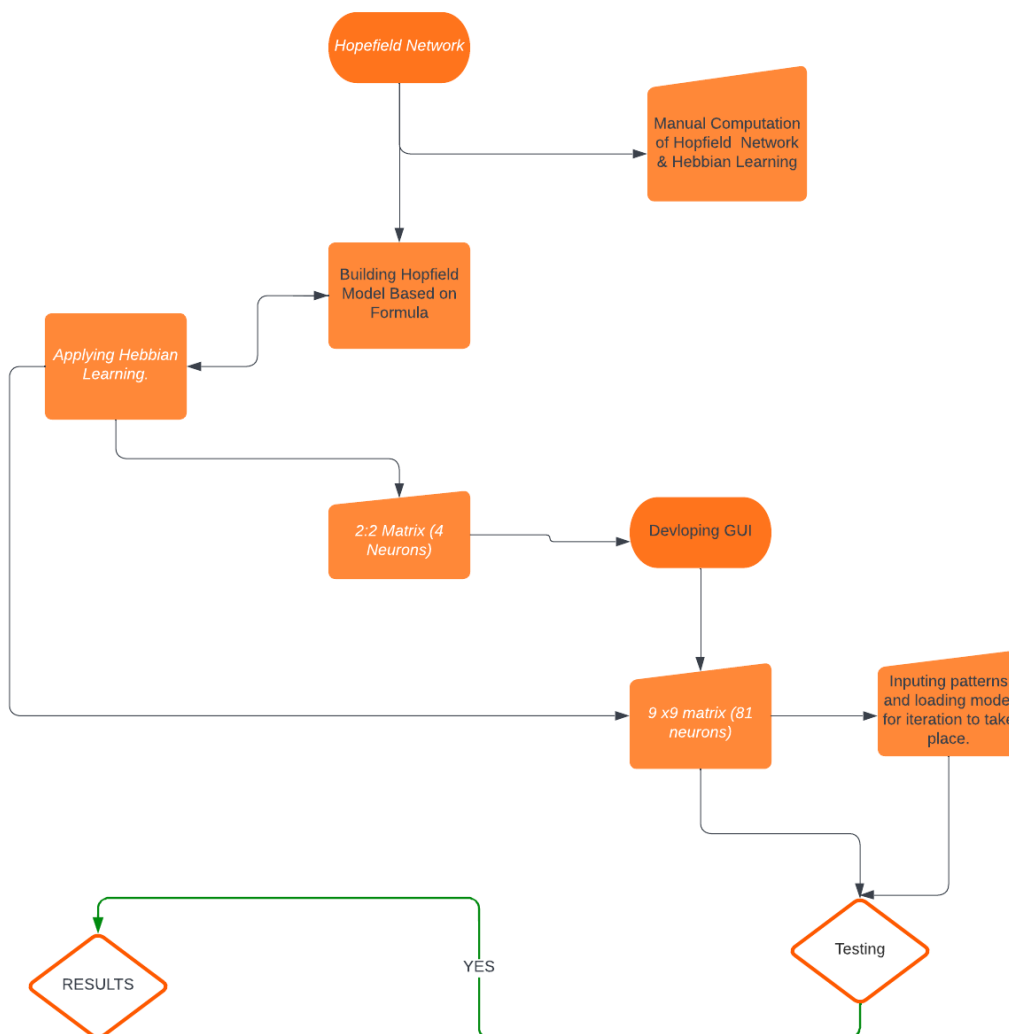


Fig 3.9

CHAPTER 4

ANALYSIS AND DISCUSSION

This chapter centers on analyzing the results acquired based on tests done in the previous chapter. The discussion would also cover the experiments and gives reasons as to why certain observations were deduced based on the results acquired.

4.1 ANALYSIS OF RESULTS

The main objective of this research involved developing a predictive Hopfield Network model to help solve optimization problems such as pattern recognition. Upon development of this model, and during implementation carried out and highlighted in the section above, one thing can be deduced from the model. From the tests above, it was deduced that the model accurately stores images/patterns given to it. Secondly, the model does well in predicting or returning the expected output pattern once the guessed pattern has been applied to it. As such, it can be deduced that the model stores the input images/patterns, applies the learning rules in recognizing the patterns from the guessed pattern(s), and returns the output. Based on regular iteration which is carried out in the model. The model learns each pattern, fire, and unfired neuron and then adequately stores this in its memory. As such, as seen in the first test where the model returns the stored image based on the guessed pattern. In the second test, the model also returned the stored image irrespective of the guessed pattern which wasn't similar to the input pattern stored within the memory of the model. Thus, the Hopfield Network model in hindsight does a good job of receiving an input pattern/storage, storing it, and returning the stored pattern/image based on the guessed pattern applied to the model. As such we can deduce just yet that the model satisfies the major objective of this research which is to develop a model that predicts and recognizes output patterns based on input and guessed patterns received.

4.2 DISCUSSION

The model based on the test was effective in adequately predicting and returning output images/patterns that have been stored within the memory of the model. However, based on the experiments that were performed to check the effectiveness of the model, some key insights were deduced. The model in *Experiment I*, returned an empty grid failing to and based on the guessed pattern which was similar to the first input pattern. In *Experiment II*, the model returned the first pattern/image saved even as the guessed pattern was an empty grid. In *Experiment III*, the model stored two(images) with a guess pattern not similar to the input

patterns. The output was the first pattern stored in the model. From these three experiments, we can see that the model sort of returns an output based on either an input that it stored or based on its creativity. This experiment shows the growing perception that Hopfield networks work closely and similarly to how the human brain works. The Hippocampus is; the storage of episodic memories, memories that are from an individual's distant past coupled with a more present past. When trying to remember an event, signals and sensory information are sent and this is done from the Hippocampus. In this Hippocampus, a sensory cue is present which helps an individual remember certain things that have happened. This is however similar to what happens with a Hopfield net[11]. Within a Hopfield net sensory cue which could be clicking a particular box within the grid signals the memory on something similar being stored within, this then leads to the model returning it as an output. Looking back at the experiments, in *Experiment III*, similar 'sensory cues' are clicked thus leading to why the model returns the first input as the output.

With *Experiments I & II*, just as the brain can sometimes "create images or memory of what it appears we have experienced or seen, the Hopfield net also has that ability. In the first experiment, in particular, the model returns an empty grid as the guessed pattern has just one neuron fired. This could be because the memory assumed and returned an output based on the location of the fired neuron. In the second experiment, the model simply returned the first image/pattern stored in its memory. Possibly insinuating that the model asides from "creating output images" only return the first input pattern/image as no pattern was guessed.

In *experiments IV & V* the results show the existence of Spurious minima. Spurious minima which in this case can be seen as patterns that were creatively memorized by the network model. These created patterns have similar combinations of patterns which we wanted to be learned. Mathematically, this is the local minima present within the energy functions implying that if the network is set with values, it can evolve through the local minima trapping it[11]. Also, increasing the number of patterns willing to be stored especially if the patterns aren't similar were issues as the model returned just empty grids. This could be because research into recurrent neural networks in this case Hopfield nets memory has shown a limit to how large the memory is. Retrieval errors conflict when the number of stored patterns surpasses 14% of the size of the network (nodes or neurons)[31]. This is compelling as 14% of the size of the network, which was 81 neurons is about 11.34, approximately 11. As such trying to store more than 11 patterns/images in the memory based on the network is what led to the output which was returned by the model.

The model based on testing effectively satisfied the main aims the model was developed to achieve. However, experimentation showed that numerous constraints exist which can affect how the model predicts and recognizes patterns.

CHAPTER 5

CONCLUSION AND FUTURE WORK

The aim of this research involved developing a predictive Hopfield Neural Network model that helped in solving optimization problems (pattern recognition) to be precise. Optimization problems as earlier stated are present in every scope of science and engineering as such it is important to understand and use the best possible optimized method to solve them.

During the course of this research, the Hopfield Network model was developed based on already-established literature deducing that the network was effective in solving such problems. Based on the model developed, during implementation, the model was effective to a degree in accurately predicting and returning expected output patterns based on the input and guessed patterns stored in the memory of the model. As such, pattern recognition took place, further achieving the major objectives of this research. However, when experimenting in trying to check just how effective the Hopfield Network Model is, it was understood that various constraints affect the performance of the model. Constraints such as the capacity of the model's storage, and spurious minima to name a few. It was also seen that the Hopfield Network has similar attributes based on its functions and how it works like the human brain. Generally, the Hopfield network was a good technique for trying to recognize patterns and in turn solve optimization problems. However, further work and research need to be done about the constraints which affect how the model learns, stores, predicts, and recognizes patterns.

For future work, it would be interesting to adopt other network types which give room for larger memory to memorize and store more patterns. Also, looking at not just pattern recognition but image processing where the model can try and recognize an image, maybe facial images for the fun of it.

REFERENCES

- [1] B. Müller, J. Reinhardt, and M. T. Strickland, *Neural Networks: An Introduction*. Springer Science & Business Media, 1995.
- [2] P. Picton, ‘What is a Neural Network?’, in *Introduction to Neural Networks*, P. Picton, Ed. London: Macmillan Education UK, 1994, pp. 1–12. doi: [10.1007/978-1-349-13530-1_1](https://doi.org/10.1007/978-1-349-13530-1_1).
- [3] ‘What are Neural Networks?’, Aug. 03, 2021. <https://www.ibm.com/cloud/learn/neural-networks> (accessed Oct. 11, 2022).
- [4] L. Medsker and L. C. Jain, *Recurrent Neural Networks: Design and Applications*. CRC Press, 1999.
- [5] B. Krause, ‘Optimizing and Contrasting Recurrent Neural Network Architectures’. arXiv, Oct. 16, 2015. Accessed: Oct. 19, 2022. [Online]. Available: <http://arxiv.org/abs/1510.04953>
- [6] *Hopfield Networks*, (Nov. 17, 2013). Accessed: Oct. 19, 2022. [Online Video]. Available: <https://www.youtube.com/watch?v=gfPUWwBkXZY>
- [7] F. Alemanno, M. Aquaro, I. Kanter, A. Barra, and E. Agliari, ‘Supervised Hebbian Learning’. arXiv, Sep. 07, 2022. doi: [10.48550/arXiv.2203.01304](https://doi.org/10.48550/arXiv.2203.01304).
- [8] J. J. Hopfield, ‘Hopfield network’, *Scholarpedia*, vol. 2, no. 5, p. 1977, May 2007, doi: [10.4249/scholarpedia.1977](https://doi.org/10.4249/scholarpedia.1977).
- [9] S. Liviu, ‘Hopfield nets and the brain’, *Medium*, Sep. 10, 2019. <https://medium.com/@serbanliviu/hopfield-nets-and-the-brain-e5880070cdba> (accessed Oct. 26, 2022).
- [10] J. Bemley, ‘Hopfield networks’, in *IJCNN’99. International Joint Conference on Neural Networks. Proceedings (Cat. No.99CH36339)*, Jul. 1999, vol. 6, pp. 4435–4437 vol.6. doi: [10.1109/IJCNN.1999.830885](https://doi.org/10.1109/IJCNN.1999.830885).

- [11] G. Villarrubia, J. F. De Paz, P. Chamoso, and F. D. la Prieta, 'Artificial neural networks used in optimization problems', *Neurocomputing*, vol. 272, pp. 10–16, Jan. 2018, doi: [10.1016/j.neucom.2017.04.075](https://doi.org/10.1016/j.neucom.2017.04.075).
- [12] W. Ongsakul and V. N. Dieu, *Artificial Intelligence in Power System Optimization*. CRC Press, 2013.
- [13] 'Optimization Problems'. http://www.otlet-institute.org/wikis/Optimization_Problems.html (accessed Nov. 15, 2022).
- [14] J. Brownlee, 'Why Optimization Is Important in Machine Learning', *Machine Learning Mastery*, Jun. 01, 2021. <https://machinelearningmastery.com/why-optimization-is-important-in-machine-learning/> (accessed Nov. 15, 2022).
- [15] 'Chapter 7: OSCILLATION RESPONSES IN A CHAOTIC RECURRENT NETWORK', p. 25, 2001.
- [16] I. Stoianov, 'Recurrent auto-associative networks and sequential processing', *IJCNN'99. International Joint Conference on Neural Networks. Proceedings (Cat. No.99CH36339)*, Jan. 1999, Accessed: Nov. 16, 2022. [Online]. Available: https://www.academia.edu/17080281/Recurrent_auto_associative_networks_and_sequential_processing
- [17] Hagner, D.G., Watta, P.B. and Hassoun, M.H., 1999. Comparison of recurrent neural networks for trajectory generation. *Recurrent Neural Networks: Design and Applications*, pp.243-276.
- [18] Sundareshan M. K. Wong Y. C. & Condarcure T. (2000). Training algorithms for recurrent neural nets that eliminate the need for computation of error gradients with application to trajectory production problem. *Recurrent Neural Networks : Design and Applications / Edited by L.r. Medsker L.c. Jain*.

- [19] T. Lin, B. G. Horne, P. Tino, and C. L. Giles, ‘Learning long-term dependencies in NARX recurrent neural networks’, *IEEE Trans Neural Netw*, vol. 7, no. 6, pp. 1329–1338, 1996, doi: [10.1109/72.548162](https://doi.org/10.1109/72.548162).
- [20] C. W. Omlin, L. Giles, and K. K. Thornber, ‘Fuzzy Knowledge and Recurrent Neural Networks: A Dynamical Systems Perspective’, in *Hybrid Neural Systems*, Berlin, Heidelberg, 2000, pp. 123–143. doi: [10.1007/10719871_9](https://doi.org/10.1007/10719871_9).
- [21] D. Bouchard, A. Chikhani, V. I. John, and M. M. A. Salama, ‘Applications of Hopfield neural networks to distribution feeder reconfiguration’, in *[1993] Proceedings of the Second International Forum on Applications of Neural Networks to Power Systems*, Apr. 1993, pp. 311–316. doi: [10.1109/ANN.1993.264329](https://doi.org/10.1109/ANN.1993.264329).
- [22] K.-S. Cheng, J.-S. Lin, and C.-W. Mao, ‘The application of competitive Hopfield neural network to medical image segmentation’, *IEEE Transactions on Medical Imaging*, vol. 15, no. 4, pp. 560–567, Aug. 1996, doi: [10.1109/42.511759](https://doi.org/10.1109/42.511759).
- [23] A. A. Smeda and M. E. El-Hawary, ‘Application of Hopfield neural network in routing for computer networks’, in *Engineering Solutions for the Next Millennium. 1999 IEEE Canadian Conference on Electrical and Computer Engineering (Cat. No.99TH8411)*, May 1999, vol. 1, pp. 145–149 vol.1. doi: [10.1109/CCECE.1999.807186](https://doi.org/10.1109/CCECE.1999.807186).
- [24] H. Liu and Y. Hu, ‘An Application of Hopfield Neural Network in Target Selection of Mergers and Acquisitions’, in *2009 International Conference on Business Intelligence and Financial Engineering*, Jul. 2009, pp. 34–37. doi: [10.1109/BIFE.2009.17](https://doi.org/10.1109/BIFE.2009.17).
- [25] Q. Lai, Z. Wan, H. Zhang, and G. Chen, ‘Design and Analysis of Multi scroll Memristive Hopfield Neural Network With Adjustable Memductance and Application to Image Encryption’, *IEEE Transactions on Neural Networks and Learning Systems*, pp. 1–14, 2022, doi: [10.1109/TNNLS.2022.3146570](https://doi.org/10.1109/TNNLS.2022.3146570).
- [26] R. Sari Hartati and M. E. El-Hawary, ‘A summary of applications of Hopfield neural network to economic load dispatch’, in *2000 Canadian Conference on Electrical and Computer*

Engineering. Conference Proceedings. Navigating to a New Era (Cat. No.00TH8492), May 2000, vol. 2, pp. 707–711 vol.2. doi: [10.1109/CCECE.2000.849556](https://doi.org/10.1109/CCECE.2000.849556).

[27] G. I. Kechriotis and E. S. Manolakos, ‘Hopfield neural network implementation of the optimal CDMA multiuser detector’, *IEEE Transactions on Neural Networks*, vol. 7, no. 1, pp. 131–141, Jan. 1996, doi: [10.1109/72.478397](https://doi.org/10.1109/72.478397).

[28] M. Egmont-Petersen, D. de Ridder, and H. Handels, ‘Image processing with neural networks—a review’, *Pattern Recognition*, vol. 35, no. 10, pp. 2279–2301, Oct. 2002, doi: [10.1016/S0031-3203\(01\)00178-9](https://doi.org/10.1016/S0031-3203(01)00178-9).

[29] N. Soni, E. Sharma, and A. Kapoor, ‘Application of Hopfield Neural Network for Facial Image Recognition’, Jun. 2019.

[30] D. Team, ‘Introduction to Learning Rules in Neural Network’, *DataFlair*, Jul. 25, 2017. <https://data-flair.training/blogs/learning-rules-in-neural-network/> (accessed Dec. 15, 2022).

[31] V. Folli, M. Leonetti, and G. Ruocco, ‘On the Maximum Storage Capacity of the Hopfield Model’, *Frontiers in Computational Neuroscience*, vol. 10, 2017, Accessed: Dec. 17, 2022. [Online]. Available: <https://www.frontiersin.org/articles/10.3389/fncom.2016.00144>

APPENDICES

This section provides images of the code used through the cause of the program. Appendix I is the code used in the computation of the 2 x 2 matrix. II is the code used in drawing each cell in the GUI grid. III is the code used in drawing the grid and mouse click and IV is the main Hopfield Network code and V is the code for drawing the output grid.

I

```
#Importing the necessary libraries and packages in python.
from tkinter import *
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import colors

#Assigning the number of neurons as well as the rows and columns for the matrix.
rows = 2
cols = 2
N = rows * cols

nrows = []
ncols = []

for n in range(N):
    if n+1 <=2:
        nrows.append(1)
    else:
        nrows.append(-1)
    #print(nrows)

#Applying the Hebbian Learning based on the formula to develop the matrix.
half_length = len(nrows)// cols
first_half, second_half = nrows[:half_length], nrows[half_length:]

new_list = [first_half, second_half]

p1 = np.array(nrows).reshape(-1,rows)
print(p1)

p1 = p1.reshape(-1)
p1 = p1.reshape(N,1)

print("Transpose Function\n", p1*p1.T)
print("Identity Function\n", np.identity(N))

#Calculation of weights
W = p1*p1.T-np.identity(N)
print("Weight\n", W)

#Calculation of the weighted sum to get the sign of the neurons.
x = np.array(nrows).reshape(-1,rows)
x = x.reshape(-1)
x = x.reshape(N,1)

z = np.zeros(N)
for i in range(N):
    for j in range(N):
        if j != i:
            z[i] += W[i,j]*x[j]

x = np.sign(z)
print("Weighted Sum\n", z)
print('Sign\n', x)
```

```

#Code to develop each cell in the grid for the GUI

class Cell():
    FILLED_COLOR_BG = "blue"
    EMPTY_COLOR_BG = "white"
    FILLED_COLOR_BORDER = "black"
    EMPTY_COLOR_BORDER = "black"

    def __init__(self, master, x, y, size):
        """ Constructor of the object called by Cell(...) """
        self.master = master
        self.abs = x
        self.ord = y
        self.size= size
        self.fill= False

    def _switch(self):
        """ Switch if the cell is filled or not. """
        self.fill = not self.fill

    def draw(self):
        """ order to the cell to draw its representation on the canvas """
        if self.master != None :
            fill = Cell.FILLED_COLOR_BG
            outline = Cell.FILLED_COLOR_BORDER

            if not self.fill:
                fill = Cell.EMPTY_COLOR_BG
                outline = Cell.EMPTY_COLOR_BORDER

            xmin = self.abs * self.size
            xmax = xmin + self.size
            ymin = self.ord * self.size
            ymax = ymin + self.size

            self.master.create_rectangle(xmin, ymin, xmax, ymax, fill = fill, outline = outlin

```

```

#Code to draw the grid which is used as the GUI.
class CellGrid(Canvas):
    def __init__(self, master, rowNumber, columnNumber, cellSize, *args, **kwargs):
        Canvas.__init__(self, master, width = cellSize * columnNumber , height = cellSize * ro

        self.cellSize = cellSize

        self.grid = []
        for row in range(rowNumber):

            line = []
            for column in range(columnNumber):
                line.append(Cell(self, column, row, cellSize))

            self.grid.append(line)

        #memorize the cells that have been modified to avoid many switching of state during mo
        self.switched = []

        #bind click action
        self.bind("<Button-1>", self.handleClick)
        #bind moving while clicking
        self.bind("<B1-Motion>", self.handleMouseMotion)
        #bind release button action - clear the memory of midified cells.
        self.bind("<ButtonRelease-1>", lambda event: self.switched.clear())

        self.draw()

    def draw(self):
        for row in self.grid:
            for cell in row:
                cell.draw()

    def _eventCoords(self, event):
        row = int(event.y / self.cellSize)
        column = int(event.x / self.cellSize)
        return row, column

#Code used in computing the mouse click.
    def handleClick(self, event):
        global net
        row, column = self._eventCoords(event)
        cell = self.grid[row][column]
        cell._switch()
        cell.draw()
        #add the cell to the list of cell switched during the click
        self.switched.append(cell)

        net[row,column] = (net[row,column] + 1)%2

```

IV

```

##Assigning the number of neurons as well as the rows and columns for the matrix.
rows = 9
cols = 9

N = rows * cols
stored_images = 1
print('Give me ',stored_images, 'image(s)')

#Applying the Hebbian Learning based on the formula to develop the matrix.
#Updating the synaptic weights.
W = np.zeros((N,N))
for i in range(stored_images):
    net = np.zeros((rows,cols))
    app = Tk()
    grid = CellGrid(app, rows, cols, 50)
    grid.pack()
    app.mainloop()

    p = func(net)
    p = p.reshape(-1)
    p = p.reshape(N,1)
    W += p*p.T-np.identity(N)

W = W/N

print('')

net = np.zeros((rows,cols))
app = Tk()
grid = CellGrid(app, rows, cols, 50)
grid.pack()
app.mainloop()

print('')

##Calculation of the weighted sum to get the signs of the neurons.
x = func(net)
print(x)
x = x.reshape(-1)
x = x.reshape(N,1)

z = np.zeros(N)
for i in range(N):
    for j in range(N):
        if j != i:
            z[i] += W[i,j]*x[j]

x = np.sign(z)

print(z)

print('')

x = x.reshape(rows,cols)
print(x)
output_grid(x)

```

```

#Code for the output grid.

def output_grid(X,num='f'):
    rows = X.shape[0]
    cols = X.shape[1]

    fig, ax1 = plt.subplots()
    my_cmap = colors.ListedColormap(['white', 'blue'])
    ax1.matshow(np.reshape(X, (rows,cols)), cmap=my_cmap)

    x_ticks = np.arange(-0.5,cols)
    y_ticks = np.arange(-0.5,rows)

    ax1.set_xticks(y_ticks)
    ax1.set_yticks(y_ticks)
    ax1.grid(which='major', alpha=1, color='black', linewidth=1.5)

    for tick in ax1.xaxis.get_major_ticks():
        tick.tick1line.set_visible(False)
        tick.tick2line.set_visible(False)
        tick.label1.set_visible(False)
        tick.label2.set_visible(False)

    for tick in ax1.yaxis.get_major_ticks():
        tick.tick1line.set_visible(False)
        tick.tick2line.set_visible(False)
        tick.label1.set_visible(False)
        tick.label2.set_visible(False)

    fig.tight_layout()
    fig.savefig('grid_plot'+str(num)+'.png')

```